॥ শ্রী গণেশায় নমঃ ॥

TCP: Transmission Control Protocol

- TCP works at the TL
- byte stream protocol: every byte is numbered
- TCP works end to end (process to process)

(TL)

IP is packet stream

IP: host to host (TL)
DLL: hop to hop

TCP header:

source port (16)								destination port (16)
sequence number (32)								
acknowledgement number (32)								
header length (4)	6 bits reserved	U R G	A C K	P S H	R S T	S Y N	F I N	window size (or) advertisement window (16)
checksum (16) bits								urgent pointer (16)
options (0 - 40 Bytes)								
Data (...)								



fixed header 20B

options (0-40)B

∴ header size = 20-60 Bytes (max 60 B)

Header length field = 4 bits. max = 1111 i.e. 15

∴ Scaling factor = $\frac{60}{15} = 4$.

Source Port and Destination Port: 16 bits

Every process is identified by a port #
Multiplexing and demultiplexing are done using port #s



IP address is given by DNS server.

Every well known service will have a fixed port #.
Say google.com would use HTTP (port # 80)

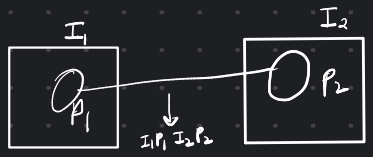
E.g.
HTTP: 80; FTP: 21; Telnet: 23; SMTP: 25;

Port # ranges:

0 - 1023	well known ports
1024 - 49151	reserved (for future)
49152 - 65535	general public

Port #s are assigned by the OS. We can also change them.

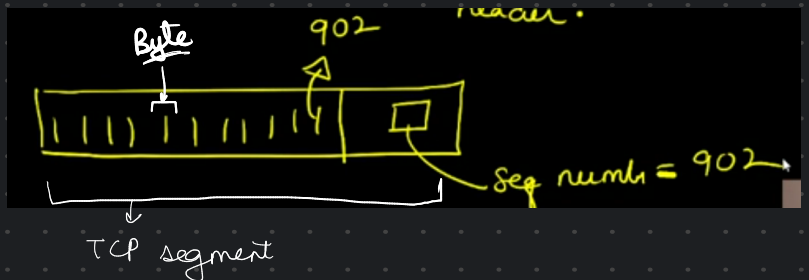
Socket: IP + Port used to identify connections
48 = 32 + 16 bits



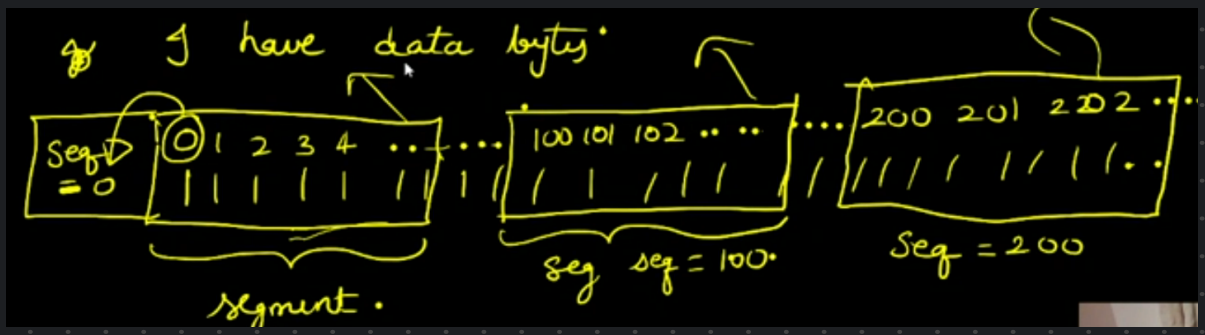
Sequence #s:

TCP is a byte-stream protocol. Every byte is numbered.

The seq # field tells the seq # of the first byte.

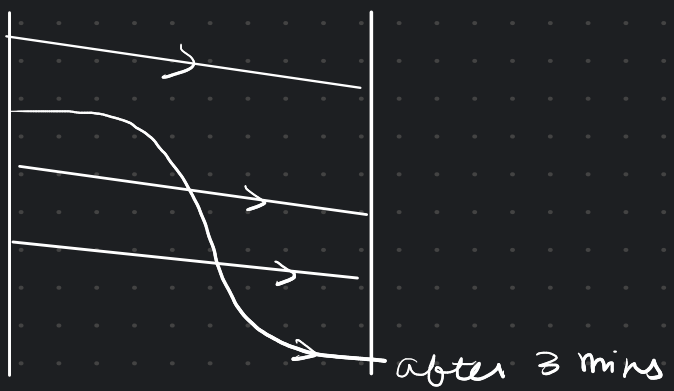


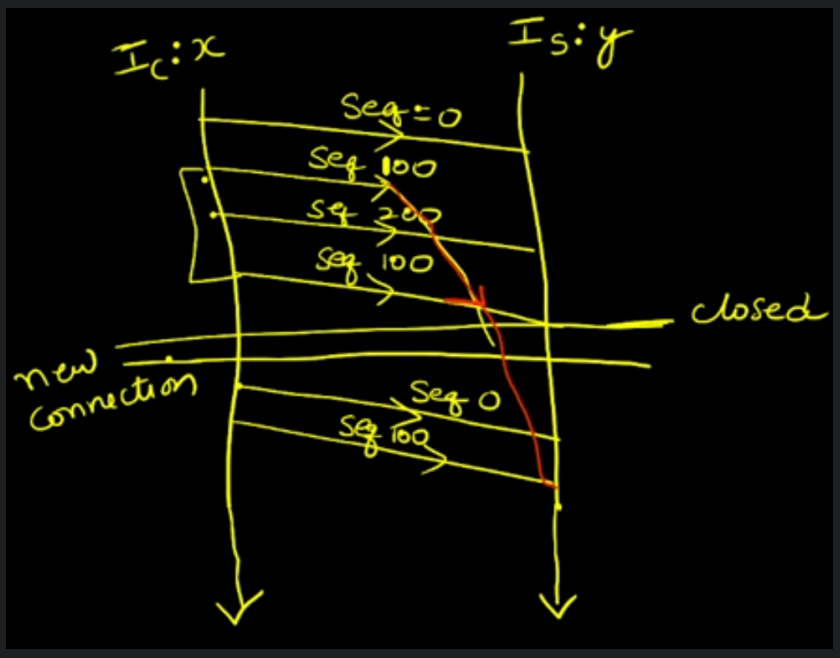
Problem:
size of seq # = 32 bits; So #s possible are 0, 1, ..., 2^32 - 1; (i.e. 2^32 #s)
(i.e. 4G numbers)



But data will be split into segments while sending
diff segments will have diff sequence #s.

In the internet, lifetime of a packet is 3 mins.
In the worst case, the packet may come after 3 mins.





a packet from the old connection may be accepted as one from the new packet.

Both have the same socket, packets have the same seq #s. So there's no way for us to know the difference.

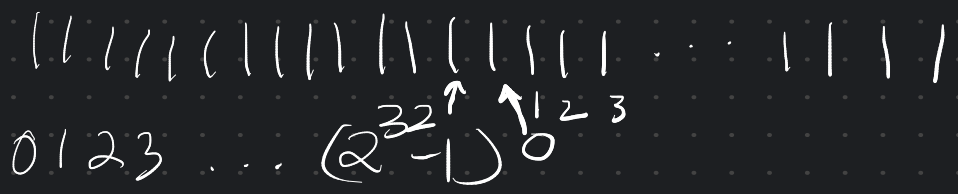
Every time we start from 0. In a new connection (after 3 mins), we start from 0 once more.

So we could just start from some random # in the new connection.

There's still a tiny chance that the #s will be repeated.

Wrap Around Time:

A TCP sender has data.
We # the bytes.



All the sequence #s are over by 2^32-1

After that, the #ing will begin once more from 0.
(Wrap-around)

Using repeated #s would obviously cause confusion.
(if 2 packets arrive at the same time)

Seq # field is 32 bits. So 2^32-1 seq #s are possible. (~4G)
i.e. we can only send 4G bytes, i.e. 4GB.
For more than that, wrap-around will occur.

Say bandwidth = 1 MBps (here B is bytes)
1 MBps -> in 1 sec, we can send 10^6 Bytes
1 sec -> 10^6 seq #s
1 seq # will be consumed in 1/10^6 secs i.e. 10^-6 secs

how many seq #s do we have? 2^32

So 2^32 seq #s will be consumed in 2^32/10^6 secs = 4294.96 secs.

So after 4294.96 secs, we will reuse the same seq #s (wrap-around)
But the lifetime of a packet is 3 mins = 180 secs.

So if we have a packet '0', it'll be alive for 180 secs.
If we reuse it after 4294.96 secs, there'd be no problem.

Q. If BW = 1GBps, then what is the wrap-around time?

→ $1 \text{ sec} \rightarrow 10^9 \text{ B} \rightarrow 10^9 \text{ seq \#s}$.

$\therefore 1 \text{ seq \#} \text{ in } \frac{1}{10^9} \text{ secs}$.

$\therefore 2^{32} \text{ seq. \#s in } \frac{2^{32}}{10^9} \text{ secs} = 4.2949 \text{ secs}$

so here same seq # is repeated even before old seq # is dead
wrap-around time < lifetime

for proper operation, wrap-around time < lifetime

Q. If BW = 1 GBps, lifetime = 180 secs, what is the min. # of seq #s reqd?

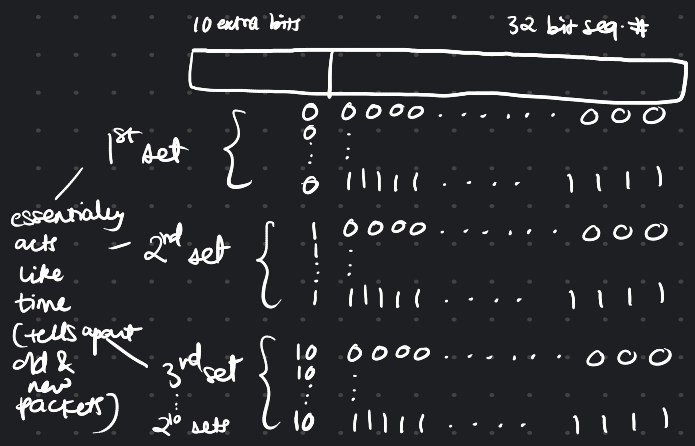
→ wrap around time > lifetime
wrap around time > 180 secs
cannot repeat seq. # within 180 secs.
So we need distinct seq #s for all bytes going out in 180 secs.

BW = 1 GBps
∴ In 1s → 10^9 B
180s → 180×10^9 B
∴ Min # of seq. #s = 180×10^9.

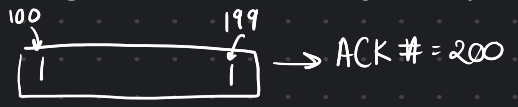
∴ # of bits in seq. # field
= $\log_2(180 \times 10^9) = 42$ bits
But we only have 32 bits.

we'll use the 10 extra bits from options.
(called timestamp)

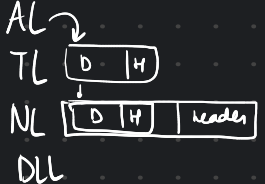
Q. Find # of bits for 10 GBps, 100 GBps.



Acknowledgement # is always seq # of next byte expected.



Seq # on a segment is always the first byte seq #



TL = IP header + TCP header + TCP data

∴ total length in IP datagram = (IP header + TCP header + TCP data) Bytes
(field in IP header)

1st byte seq # is in the header of TCP segment. How will we know last byte seq #?



data = TL - IP header - TCP header

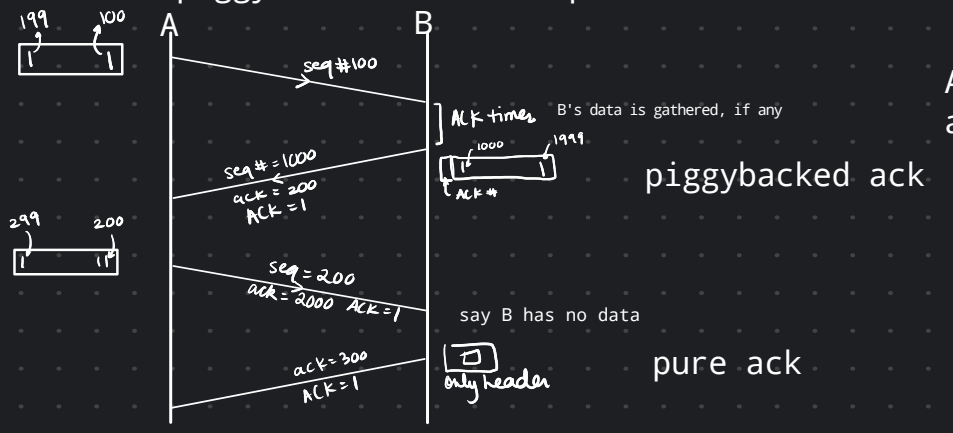
e.g. TL = 1000B, IP HL = 5B, TCP HL = 5B, seq # = 100 (first byte seq #)
so data = 1000 - 5*4 - 5*4 = 960B

(scaling factor = 4)

1st one is 100, there are 960
last byte will have seq # 100 + 960 -1 = 1059

so ACK # = last byte seq # + 1 = 1059 + 1 = 1060

TCP uses piggybacked ACKs and pur ACKs.



ACK flag indicates there is acknowledgement on the packet

Instead of sending only ack, if there's any data to send, then the ack # will piggyback on the data packet.

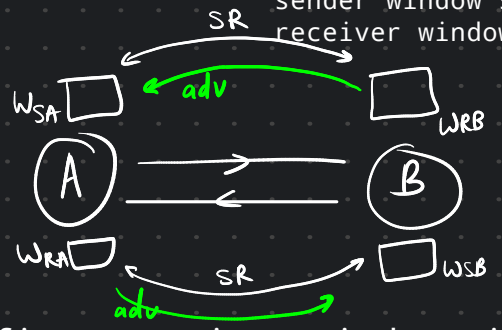
TCP is a connection-oriented protocol (resources are reserved).

3 phases:
- connection establishment (we say that we're sending data, be ready)
- data transfer (we send data)
- connection termination (free up resources)

We always start with random initial seq #s



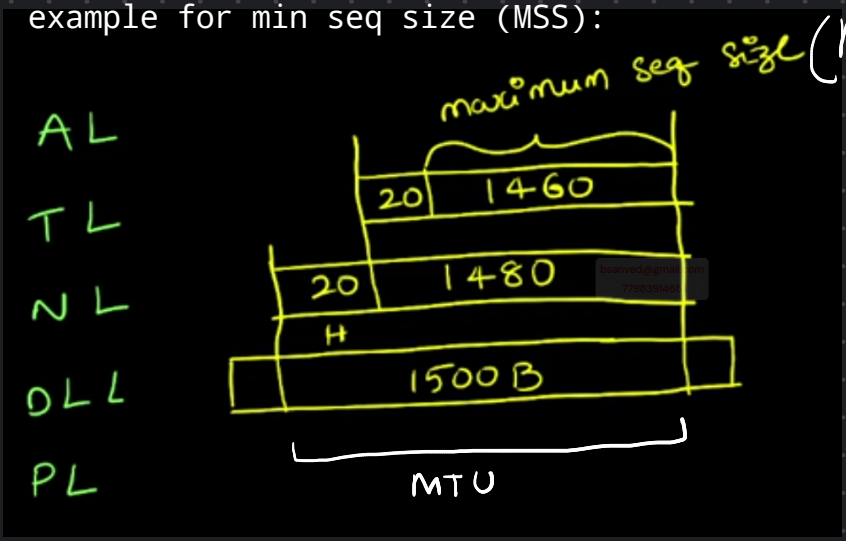
TCP is a full-duplex connection.
So A and B will both have sender and receiver windows.



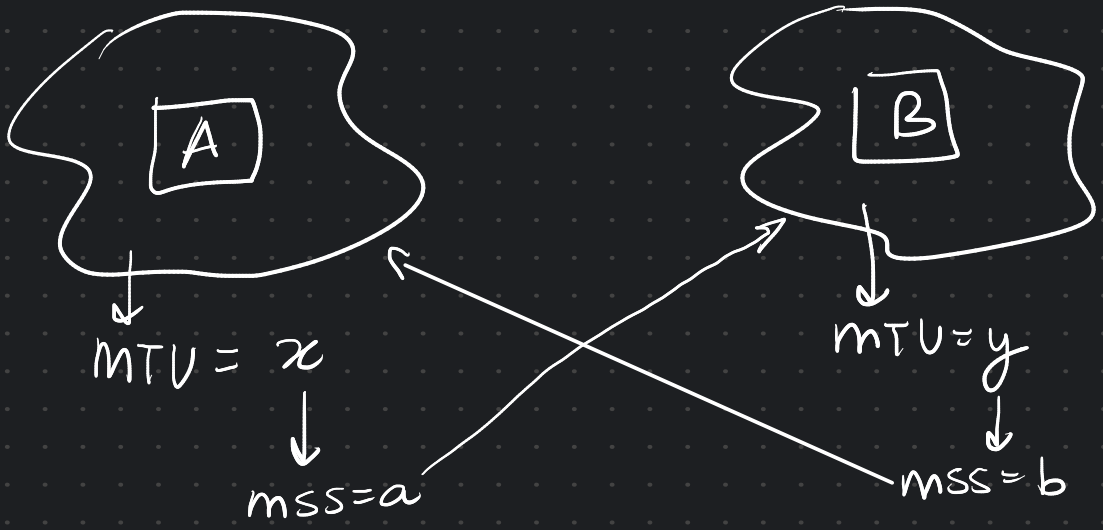
first receiver window will be decided and sent to the other side (advertisement)

sender will wait for advertisement here both A and B are both the sender and the receiver.

example for min seq size (MSS):



if anyone sends > 1460 B in 1 seg, there'll be fragmentation.



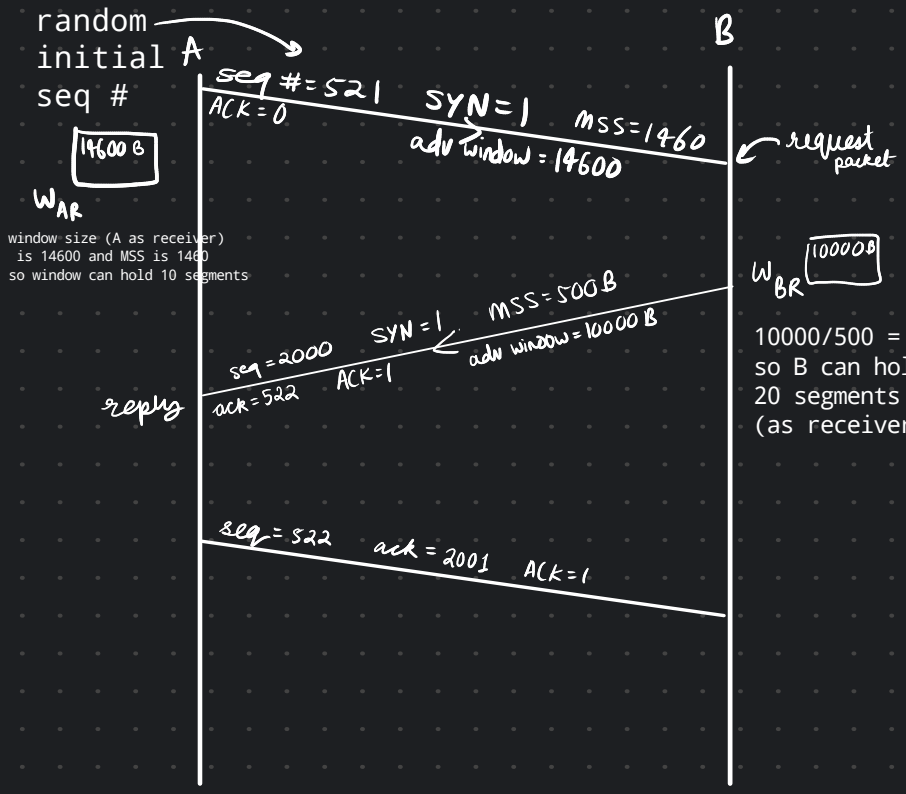
MSS is told to the other side.

so A will send segments sized <= b

B will send segments sized <= a

Connection Establishment:

SYN is the synchronization flag,
it'll specify the initial seq # (1st packet)
it'll also specify the MSS (calculated by MTU)



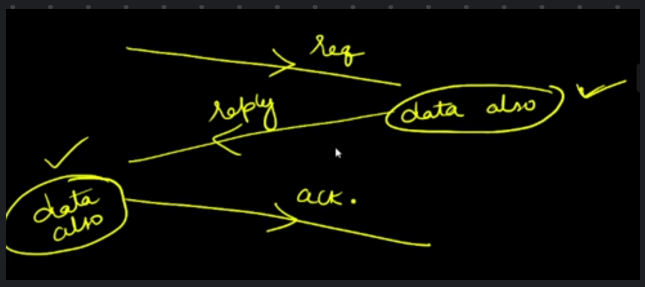
MSS and rec window size are present in options.

10000/500 = 20
so B can hold 20 segments (as receiver)

SYN packet will eat up 1 seq #
if we are sending 521 on the syn packet, then 521 is over with.
The next packet will have seq = 522

adv window will be present in every packet, mss need not be.

there can be data on the reply packet and ack



Connection establishment

Before a client attempts to connect with a server, the server must first bind to and listen at a port to open it up for connections: this is called a passive open. Once the passive open is established, a client may establish a connection by initiating an active open using the three-way (or 3-step) handshake:

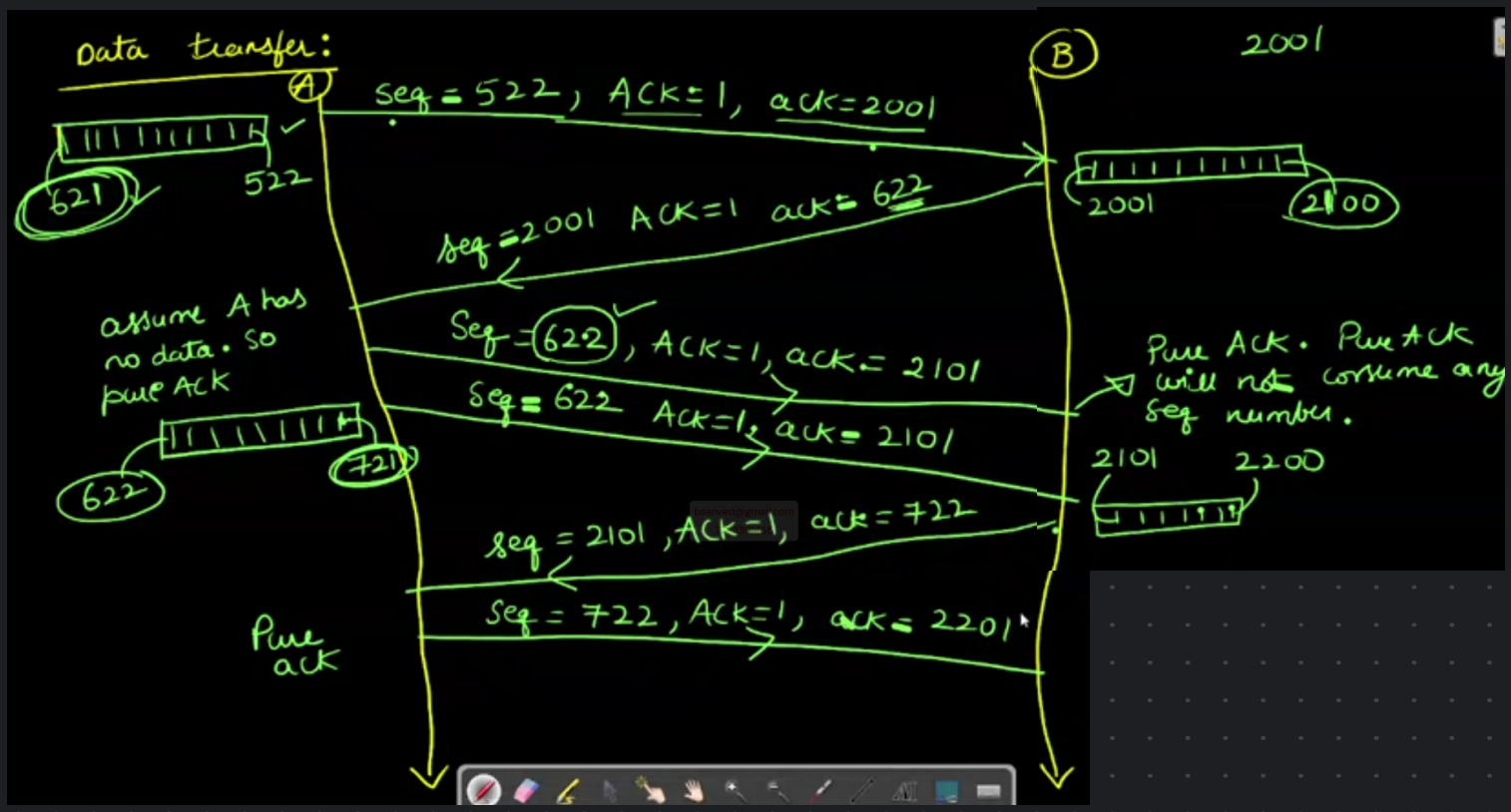
1. **SYN:** The active open is performed by the client sending a SYN to the server. The client sets the segment's sequence number to a random value A.

2. **SYN-ACK**: In response, the server replies with a SYN-ACK. The acknowledgment number is set to one more than the received sequence number i.e. A+1, and the sequence number that the server chooses for the packet is another random number, B.

3. **ACK**: Finally, the client sends an ACK back to the server. The sequence number is set to the received acknowledgment value i.e. A+1, and the acknowledgment number is set to one more than the received sequence number i.e. B+1.

Data transmission: (phase 2/3)

consider the next packets to be sent
seq #s will be 522 and 2001

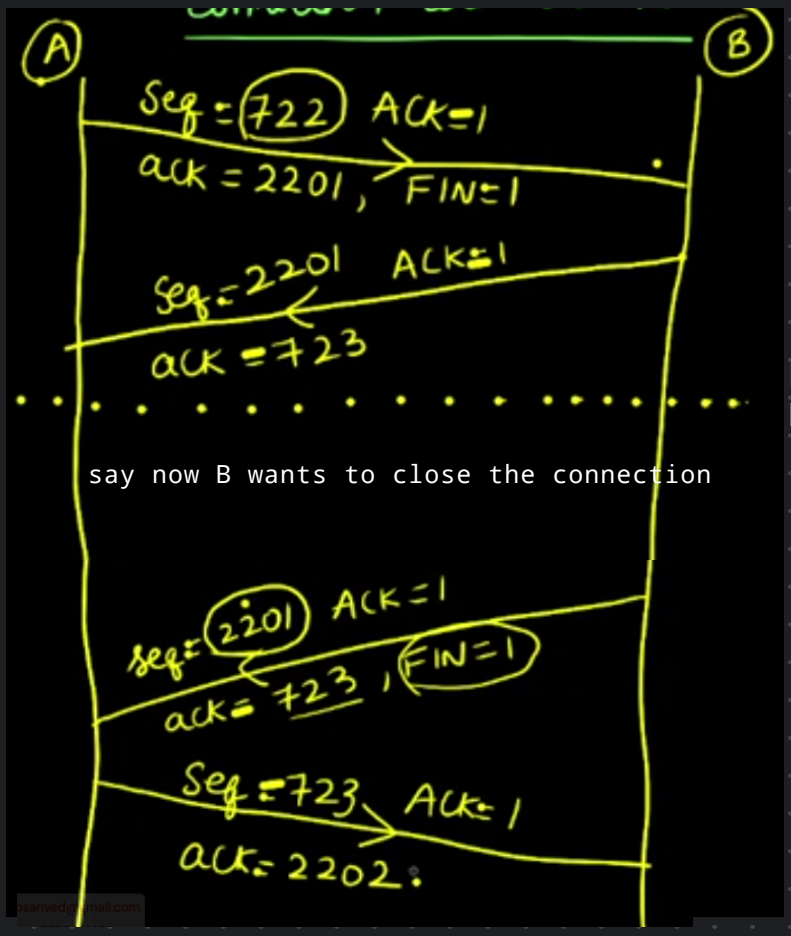


SYN	ACK	
1	0	1st segment/ req packet
1	1	2nd seg/ reply packet
0	1	acknowledgement is present (pure or piggybacking)
0	0	not possible

every segment that isn't the first will have ACK = 1

Connection Termination:

when A wants to terminate the connection (A to B), it'll send FIN = 1
FIN eats up 1 seq #



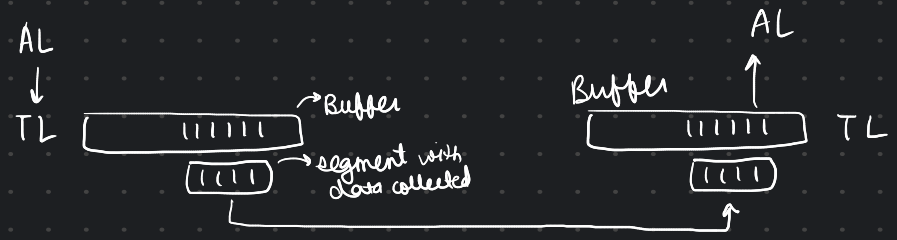
here A to B is terminated
B to A is still open

now A->B will be pure ACK
B->A will be piggyback

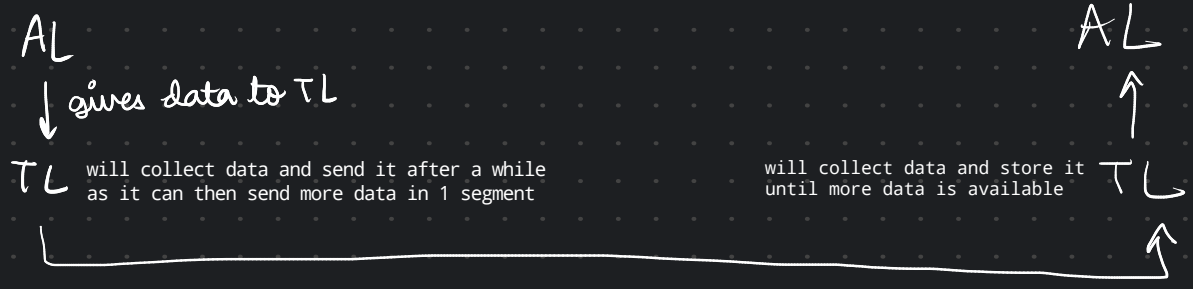
Flags in TCP: SYN, ACK, FIN, PSH, URG, RST

SYN: Synchronization
ACK: indicates if the segment contains acknowledgement
FIN: terminate the connection

PSH: (push)



TL buffers the data, not sending it immediately.
(for efficiency)



This is called buffering.

This is done for efficiency. But some applications need interactivity more than efficiency. (like chatting, remote login, etc.)

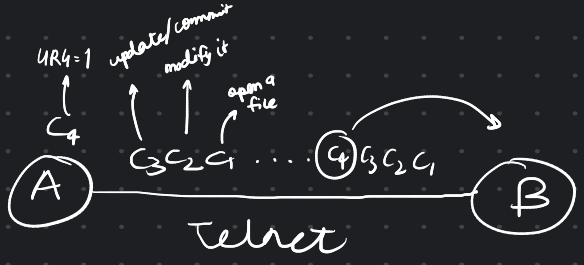
So TL shouldn't wait even if the data is too small ('hi').

PSH flag tells TL to "push" the data immediately.

obv all stuff wouldn't need PSH. We won't send too little data in say emails.

Disadvantage:

- say there's only 1B of data. TL will add 20B header. NL will add 20B header. For that, 20B ack will come. That ack will have 20B NL header. PSH is inefficient. But it is needed sometimes, so we use it.



URG: Urgent

say we remote login into a system.
We sent some commands.
open, modify, and commit changes to a file.
Say we realize after C1, C2, C3 that we fucked up.
we send C4 to say we want to disconnect. (ctrl + c)
C4 has to be processed before C1, C2, C3 are.

We'll use URG = 1

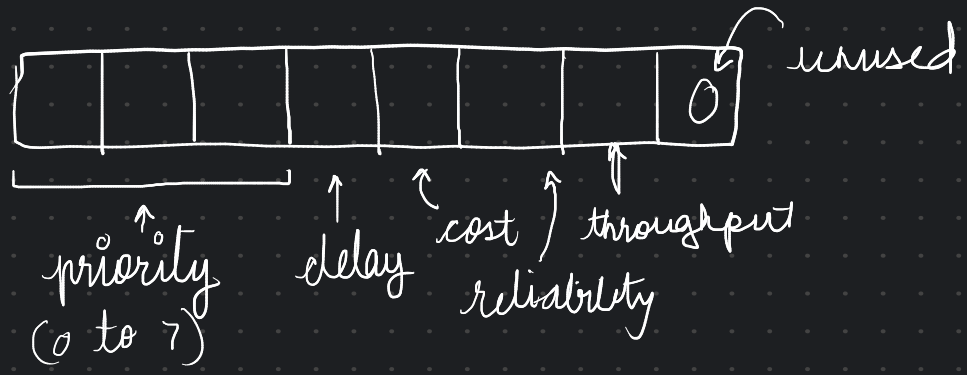
The packet will be taken out of order and given a priority.

Even intermediate routers will forward packets with URG = 1 first.

Routers don't have a TL. How would they know of the URG flag?

At the sender, in the IP header, priority is set to 7 (high)
in the Type of Service field.
Routers have NL though, so they'd get to know about it.

Type of Service in IP header (8 bits):



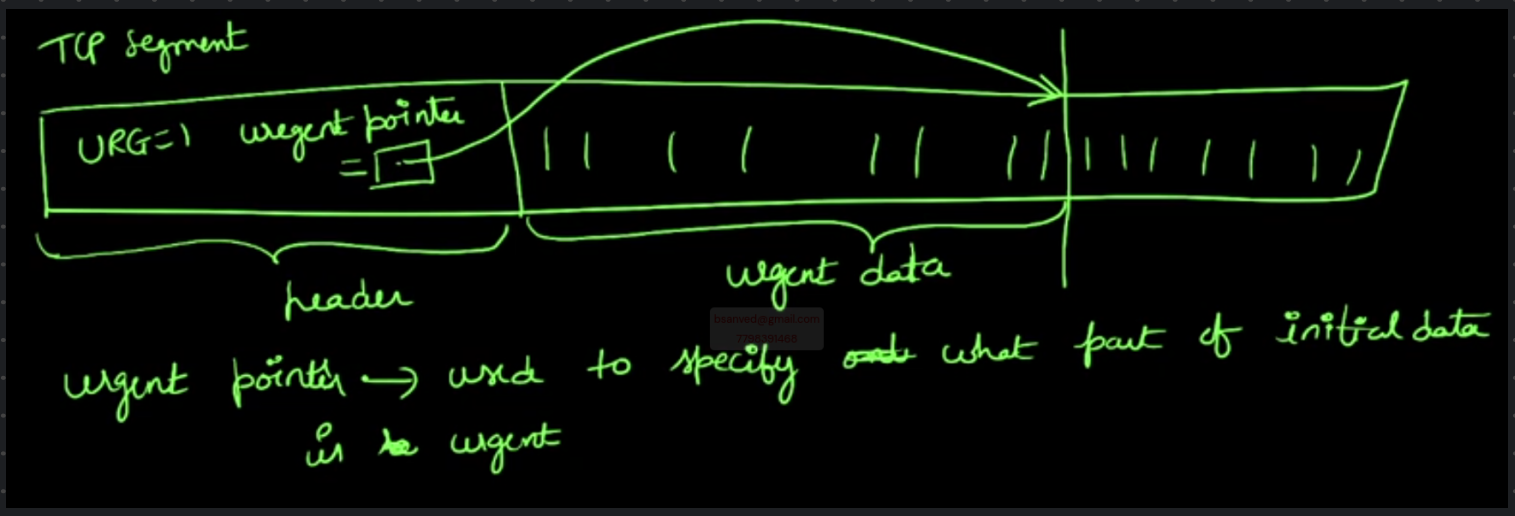
delay: 1 - low i.e. send on a route with low delay

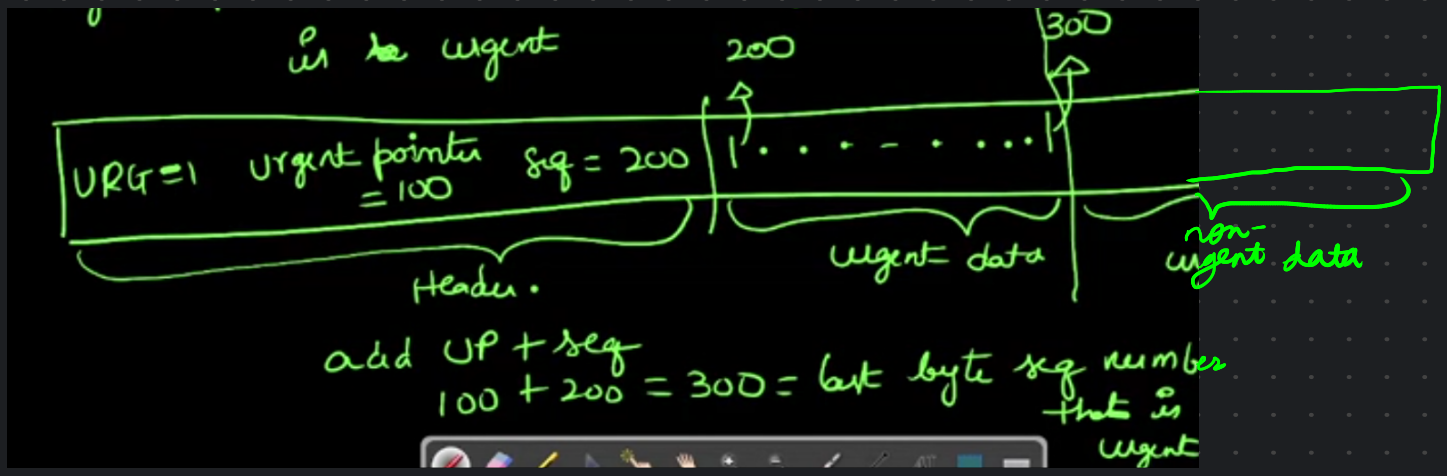
cost: 1 - send on a route with low cost

reliability: 1 - high

throughput: 1 - high

problem: everyone would put all 1's in these fields.
So it isn't allowed by the default gateway.

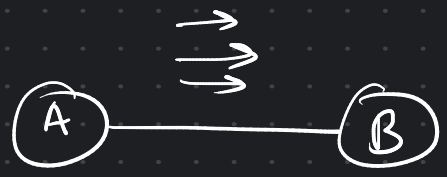




here total urgent bytes = 101

RST: reset

used to reset the connection



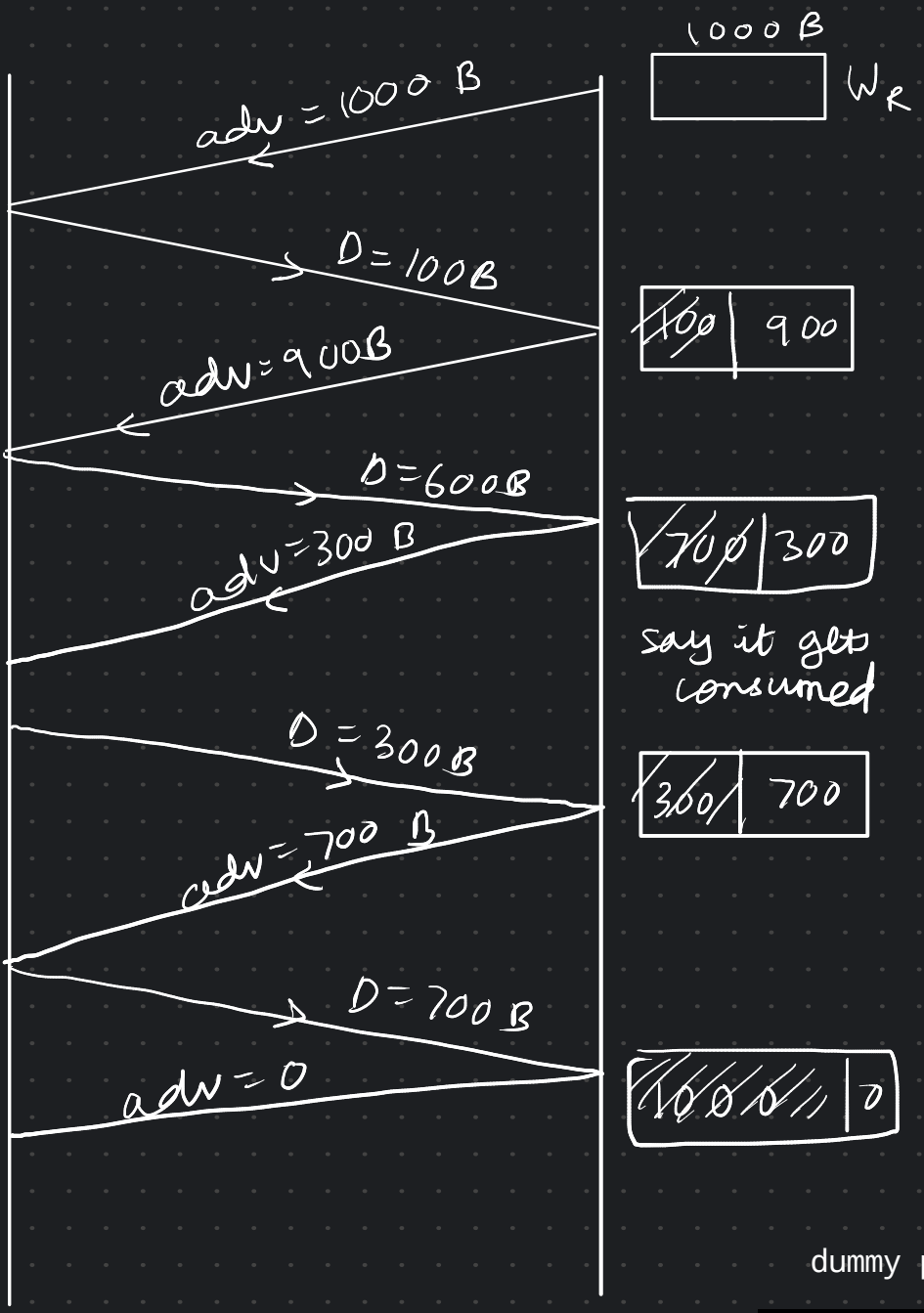
say A is sending some packets to B.
B suddenly went down and back up.
B will forget everything about the connection.
It won't understand what it is receiving.

So B will send RST = 1 to A.

Sometimes the receiver is very slow. Then RST is used to clear the buffer.

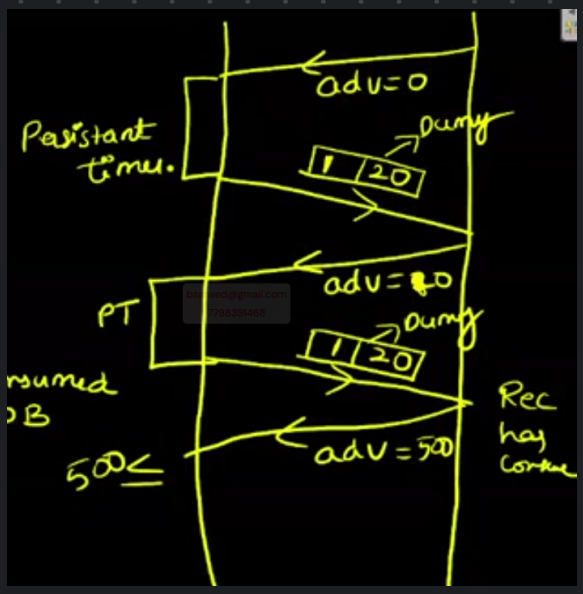
Advertisement Window:

used for flow control; present in TCP header;



when adv will be 0, a persistent timer is started.

dummy packets will be sent

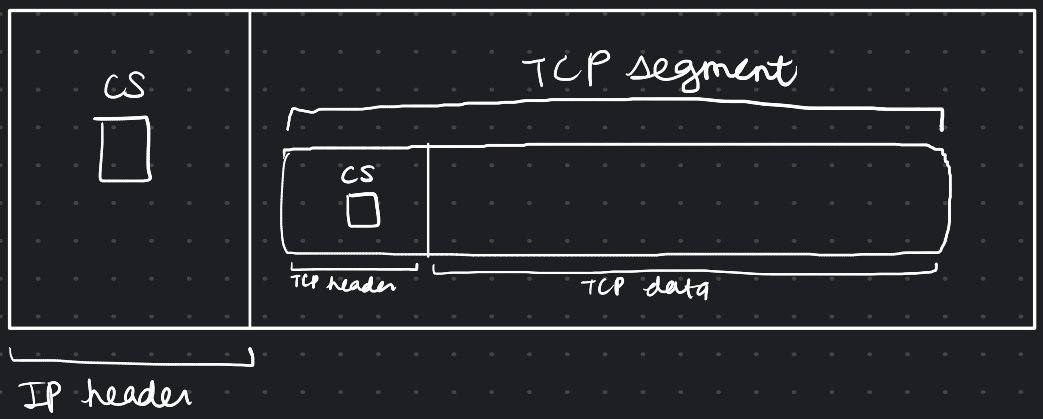


when adv becomes 0, the sender wouldn't get to know when the receiver buffer is freed.
to simply get a response, dummy packets are sent. (they only have header, and in the buffer, only the data is stored, not the header)

problem: window size or adv window is 16 bits.
So max # is 2^16 - 1, i.e. 65535
So this is too small. 16 bits is already in the adv window.
Along with it, 14 bits from the options are used to make it 30 bits.

So we can advertise as much as 2^30, i.e. 1GB.

Checksum:



IP header is calculated on IP header only
TCP header will be calculated on
pseudo IP header + TCP header + TCP data

why is checksum calculated on the IP header twice?
- to make sure

say A is writing a letter to B. A put it in an envelope. On the envelope, the recipient's house # is written. Say there's another person called B adjacent to B. If the address is mistaken as that of the other B, there'll be an error.
Instead, if the address is written inside on the letter as well, an error will be avoided.

So TCP CS is calculated on TCP H + TCP D + pseudo IP H

pseudo IP header (12 B)

SIP (32)		
DIP (32)		
00000000 (8 zeros)	Protocol (8)	TCP segment len (16)

Options:

- timestamp (when wrap-around time < lifetime
- window size or adv window extension (14 bits)
- parameter negotiation (MSS)
- padding -> TCP header has to be a multiple of 4 (scaling)

---

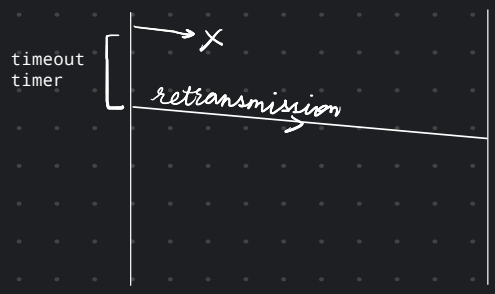
we're done with TCP header

Retransmissions in TCP:

TCP uses a combination of SR + GBN (for flow control)

Why SR?
- W_S = W_R
- out of order packets are accepted

Why GBN?
- ack are cumulative

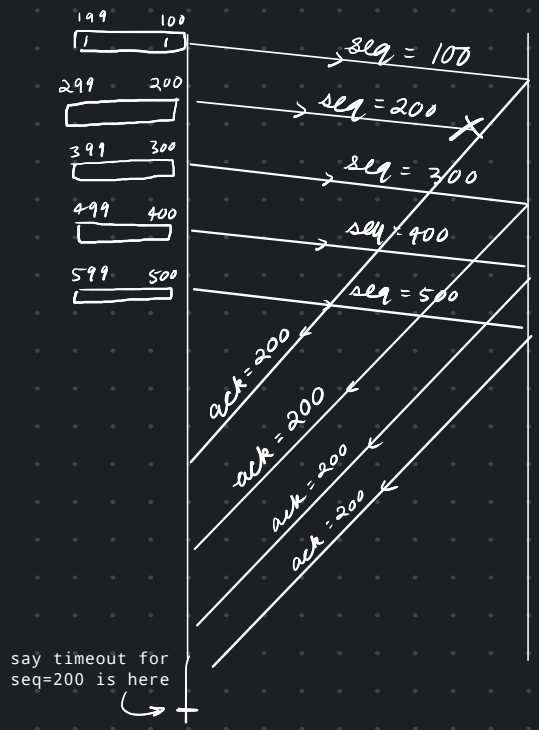


If timeout timer is too long, then unnecessary time is wasted.

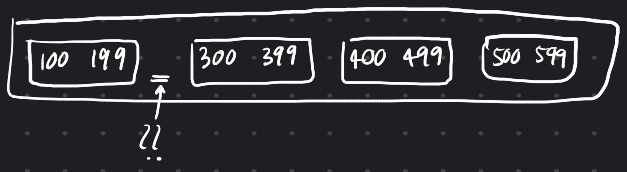
If it is too short, then unnecessary retransmissions might take place. (before ack comes back, you retransmit)

So timeout timer should be neither too long nor too short.

Can we detect a packet loss before timeout?



say seq=200 is lost



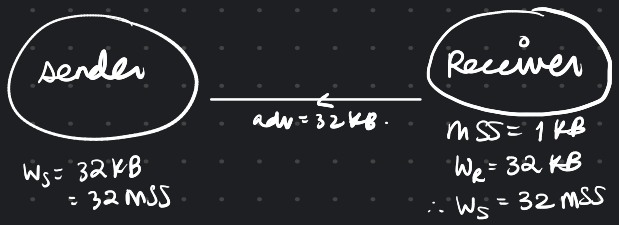
these (out-of-order) packets are accepted

But ack will be sent for seq=200
first is original, rest are duplicate.

Rule: whenever we get 3 duplicate acknowledgements, retransmit immediately.

(early ack)

TCP Congestion Control:





the n/w may not be able to handle all the traffic (32 mss) at a time

receiver capacity is known by the adv window, but n/w capacity is not known.
So we use congestion control algo.

receiver window size W_R = 32 mss
we don't know network capacity. so we'll use a congestion window.

Initially, it'll be 1 mss.

$$W_c = 1 \text{ mss}$$

$$W_s = \min(W_c, W_R) = \min(1, 32) = 1 \text{ mss}$$

Sender window will be equal to congestion window size = 1mss.
Initially it'll send 1mss. Then ack will come.
It'll send 2mss. Then ack will come.
Then 4mss, ack. Then 8 mss, ack. Then 16, then ack.

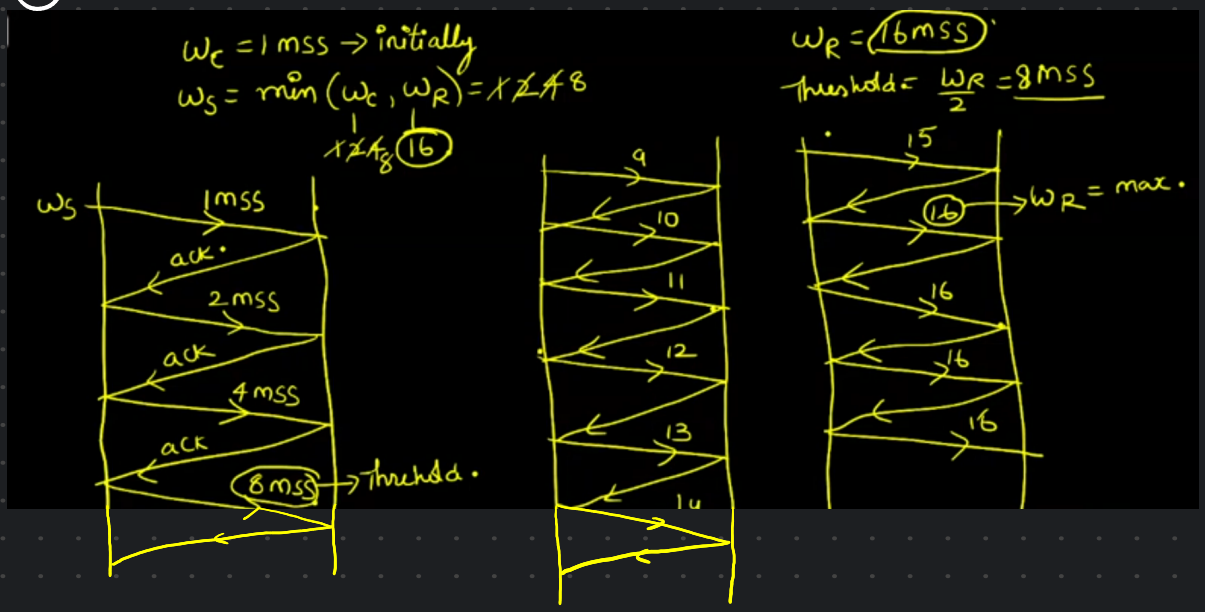
We'll grow exponentially till threshold. Threshold = $\frac{W_R}{2} = 16$

Beyond the threshold, we'll grow increasing by 1 every time.

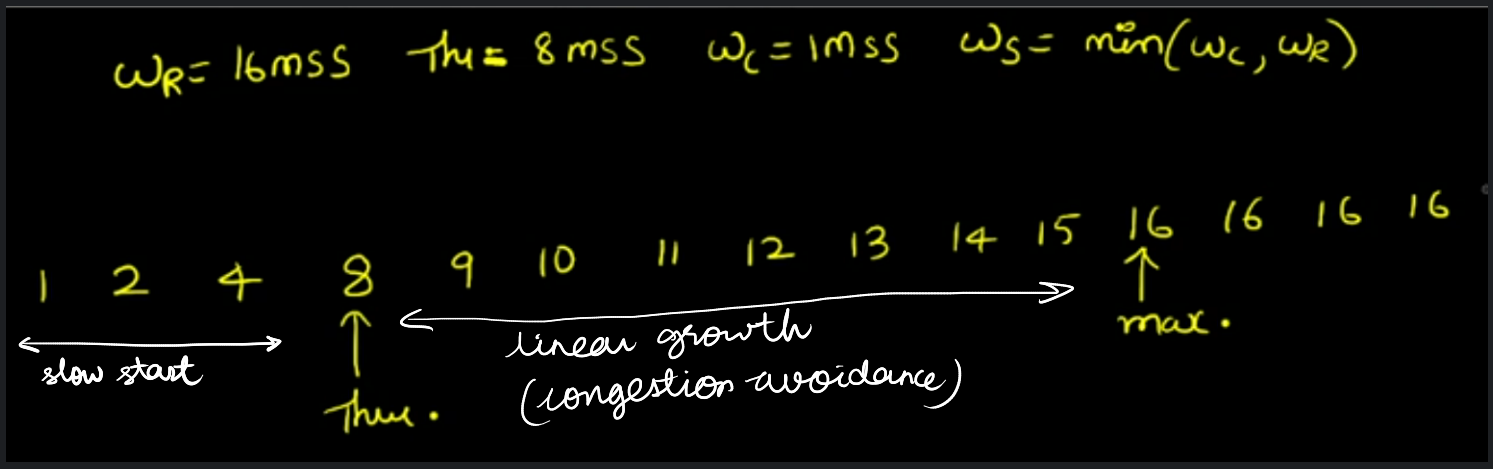
We'll send 17 mss, then 18, then 19, and so on.
This'll go on till 32 mss (W_R)

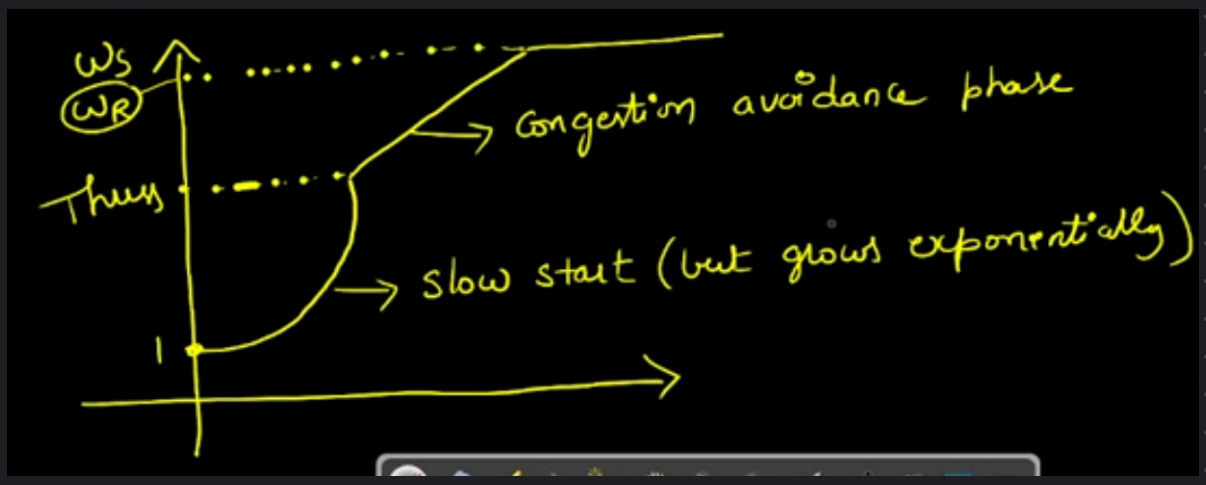
---

say $W_R = 16$ mss



After how many RTT's do we reach the full capacity?
1 2 4 8 9 10 11 12 13 14 15 16
i.e. 11 RTTs





Initially (till the threshold), we'll be careless and send exponentially growing packets.
In the congestion avoidance phase, we'll be careful.

Congestion: All the network devices are too busy. Packets will be discarded.

How will we know about congestion? When packets are lost.
How will we know if a packet is lost? 2 ways:
- timeout will occur
- next 3 packets are sent, we'll get 3 duplicate acknowledgements

The 1st case represents severe congestion. The 2nd represents a mild congestion.

(One more way to detect congestion:
- ICMP source quench.)

Congestion algo will react differently for timeout and 3 duplicate acks.

(AIMD is additive increase multiplicative decrease)

TCP Congestion Control Algo:

Phase 1: slow start
Phase 2: congestion avoidance
Phase 3: congestion detection

you can detect congestion using
- timeout (severe)
- 3 duplicate acks (mild)

Timeout:

say $W_S = 32$ mss $W_C = 1$ mss

$\therefore Th = \frac{32}{2} = 16$ mss

$W_S =$ 1 2 4 5 6 7 8 16 17 18 ↑ timeout

say there's a timeout
The new threshold will be Wc/2

threshold $= \frac{W_C}{2} = \frac{18}{2} = 9$

Algo will enter slow start

1 2 4 8 9 10 11 12 13 14 15 16 ↑ timeout

$Th = 16/2 = 8$

1 2 4 8 9 10 11 ... 32, 32, ...

3 duplicate acks:

$W_R = 32$ mss $\quad W_L = 1$ mss

$\Rightarrow Th = 16$ mss

1 2 4 8 16 17 19 20

new $Th = 10$

↑ longest congestion avoidance phase

TCP Timer Managament

i. Time-wait timer
ii. Keep alive timer
iii. Persistent Timer
iv. Acknowledgement timer

i. Time wait timer

On FIN = 1, server shouldn't close connection immediately. It has to wait for late packets. It should wait for 2 msl (max segment lifetime)

MSL can be 10 secs or 30 secs or 1 min or 2 mins or 3 mins (on implementation)

Time wait timer = 2 msl

ii. Keep alive timer:

Say there's a server having too many connections. A new connection would be rejected.
The server should close off idle connections.
To check idle connections, the keep alive timer is used.
If there isn't any activity till the keep alive timer (started from the last received packet) ends, a connection is closed.

iii. Persistent Timer:

Say the receiver advertises 100. You send 100. Then receiver would advertise 0.
Then you'd wait for a while (persistent timer), then send a dummy packet or probe to check if receiver window is freed.

iv. Acknowledgement Timer:

It is used to send ack for many packets received within that time.
It is used to implement cumulative acknowledgements.

v. Time out timer:

Generally in one hop, $T_p = \frac{d}{v}$, $RTT = 2 * T_p$, $TO = 2 * RTT$

In end to end, we don't know propogation delay (Tp) as every packet can take a diff route.

In the DLL, we take one hop. So RTT is fixed. So timeout is fixed.
In the TL, there are many hops, so RTT varies. So TO has to be dynamic.

To calculate TO, 2 algos:
- basic
- Jacobson's

Basic algo to calculate timeout:

When we send a packet, we need to est the timeout. But we don't know the RTT.
So we'll guess it.
Initial RTT = 10ms (guess)
TO = 2*RTT = 20ms.

Say the ack comes back in 15 ms. So actual RTT = 15ms.
This has happened for the first packet.

Next RTT = alpha * (Initial RTT) + (1-alpha) * (actual RTT)

$$\text{Next RTT} = \alpha(\text{IRTT}) + (1-\alpha)(\text{ARTT})$$

if alpha = 1, we are completely static
if alpha = 0, we are being too dynamic

generally 0 <= alpha <= 1

Let alpha = 0.5

So for first packet,
IRTT = 10ms, so TO = 2*IRTT = 20ms.
But ARTT = 15ms.
Next RTT = a*IRTT + (1-a)*ARTT = 0.5*10 + 0.5*15 = 12.50ms

For next packet, IRTT = 12.50ms
TO = 2*12.5 = 25ms
Say this time actual RTT turned out to be 20ms.
So next RTT = 0.5*12.5 + 0.5*20 = 16.25ms.

For next packet,
TO = 2*16.25 = 32.5ms
Say ARTT = 10ms.
next RTT = 0.5*16.25 + 0.5*10 = 13.125ms.

For next packet,
TO = 2*13.125 = 26.25ms

and so on...

We take TO = 2*RTT The 2 essentially has no meaning.
So we'll use Jacobson's.

Jacobson's algo to calculate timeout:

We make assumptions:
- IRTT = 10ms (guess)
- Initial deviation = 5ms (guess)
i.e. RTT = 10 +- 5 ms. (5 to 15 ms)

TO = 4*D + RTT D = Deviation TO = 4D + RTT
= 4*5 + 10 = 30ms

Say actual RTT turned out to be 20ms.
IRTT = 10ms
Actual Deviation = AD = |IRTT - ARTT|
AD = 10ms

Next RTT = alpha*IRTT + (1-alpha)*ARTT for 2nd packet
= 15ms assume alpha = 0.5
ND = alpha*ID + (1-alpha)*AD next deviation from initial and actual deviations
= 7.5ms

For 2nd packet,
IRTT = 15ms, ID = 7.5ms, TO = 4D + RTT = 45ms
But ARTT = 30ms
AD = ARTT - IRTT = 15ms
NRTT = 22.5ms, ND = 11.25ms for 3rd packet

For 3rd packet,
IRTT = 22.5ms, ID = 11.25ms, So TO = 4D + RTT = 67.5ms
But ARTT = 10ms
AD = 12.5ms
NRTT = 16.25ms, ND = 11.875ms for 4th packet

For 4th packet,
IRTT = 16.25ms, ID = 11.875ms, TO = 4D + RTT = 63.75ms

and so on...

Both basic and Jacobson's algo are based on ARTT (actual RTT).
i.e. we send for a packet and wait for it to come back.
That value is used in the next timeout.

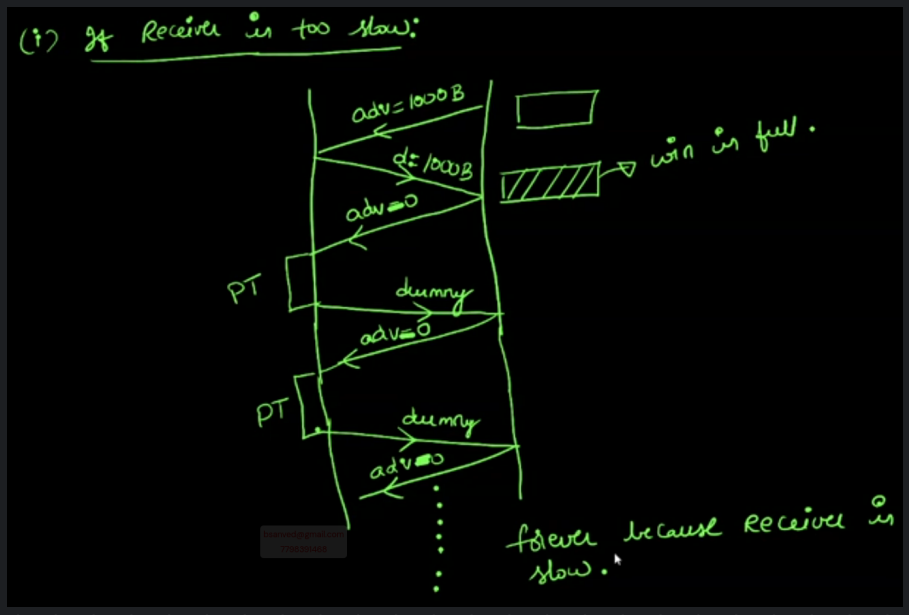
What if the packet doesn't come back?
ARTT and TO is set by Karn modification.
It says that whenever a packet is lost, just double the previous TO.

Silly Window Syndrome:

Silly window syndrome (**SWS**) is a problem in computer networking caused by poorly implemented TCP flow control. A serious problem can arise in the sliding window operation when the sending application program creates data slowly, the receiving application program consumes data slowly, or both. If a server with this problem is unable to process all incoming data, it requests that its clients reduce the amount of data they send at a time (the window setting on a TCP packet). If the server continues to be unable to process all incoming data, the window becomes smaller and smaller, sometimes to the point that the data transmitted is smaller than the packet header, making data transmission extremely inefficient. The name of this problem is due to the window size shrinking to a "silly" value.

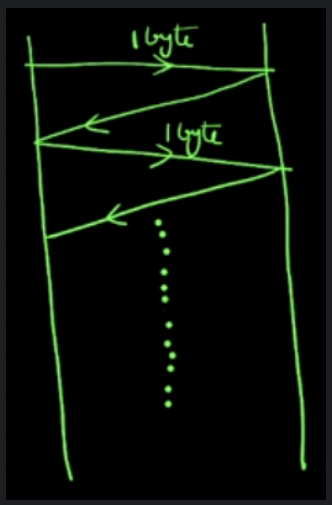
Since there is a certain amount of overhead associated with processing each packet, the increased number of packets means increased overhead to process a decreasing amount of data. The end result is thrashing.

i. receiver is too slow, and keeps getting dummy packets.



solution:
reset the connection using RST = 1

ii. sender is too slow, and is sending really small packets.

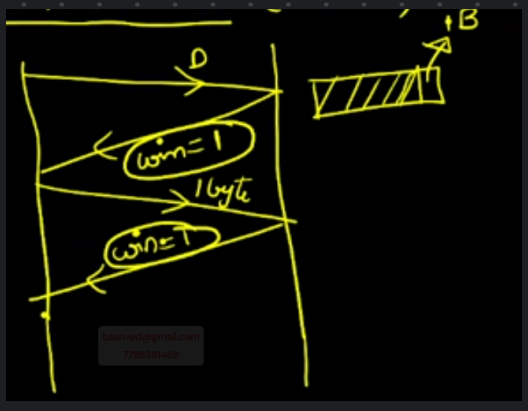


soln: Naggle's algo
It says to wait for 1 RTT and then to send all the data produced by the application layer in 1 RTT in 1 segment.

e.g. sender is producing 1B/ms and RTT is 1000 ms.
Then in 1 RTT, 1000B data is produced.
So send 1000B in 1 segment.

Condition: if we get 1 segment sized data, then send immediately.

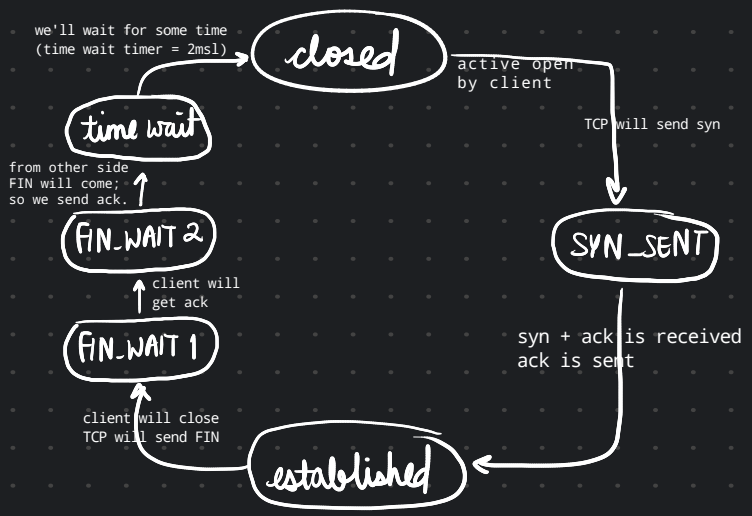
iii. receiver is too slow and we get adv = 1 every time

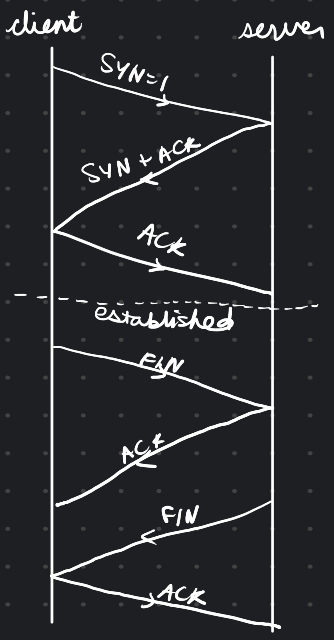


capacity is always 1B, so receiver is consuming 1B at a time.

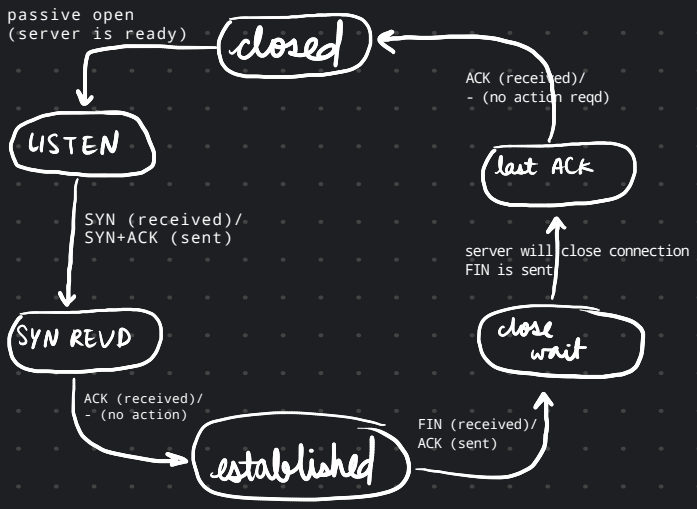
Clarke's solution:
The receiver should wait atleast till 1/2 of the buffer or 1mss, and then advertise the window.
It won't simply advertise 1B as the window size. It'll wait till more of it is freed.

Client side TCP state diagram:



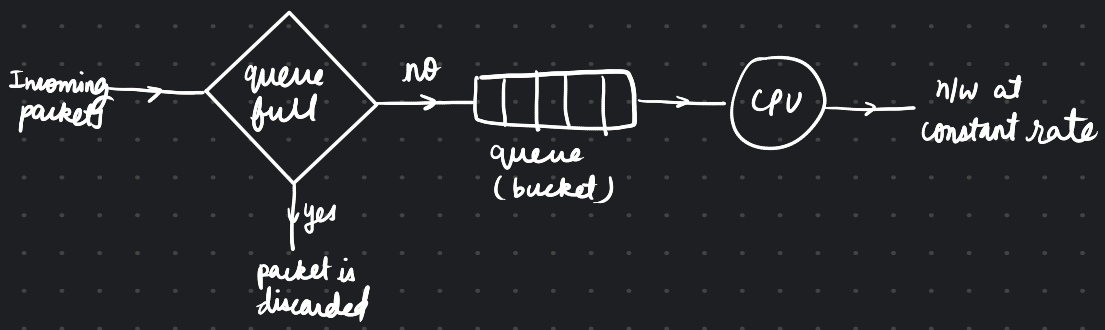


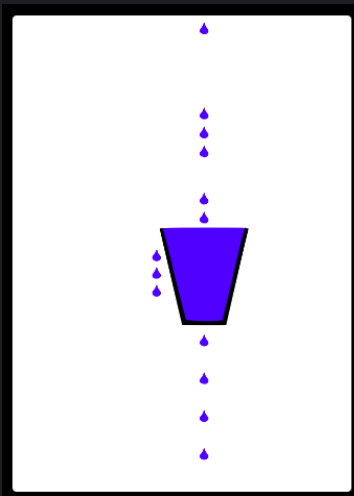
Server side TCP state diagram:



Congestion Control at IP layer:
(Traffic Shaping)

Leaky Bucket:
Think of a bucket with a little hole at the bottom.



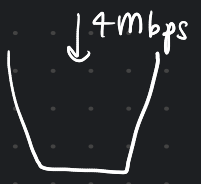


The leaky bucket analogy. Water can be added intermittently to the bucket, which leaks out at a constant rate until empty, and will also overflow when full.

Q. If inflow = 5 Mbps, outflow = 1 Mbps, burst time = 5s, then what is the size of leaky bucket?



net inflow = 5 - 1 = 4 mbps.



Burst time = 5s.

Size of bucket = 4 mbps x 5s = 20 mb.

Token Bucket:

Say in leaky bucket, there's no inflow for a while. Still, the sender cannot send more packets later.

Think of having lunch coupons for a month. Say you don't eat on a particular day, you could still use the coupon later.

In token bucket, tokens will get accumulated in the bucket.
You could use all of them at once.

Let the capacity of the token bucket be 'c' tokens (initially has c), and tokens enter the bucket
at a rate of 'r' tokens per second.
Then the max # of packets that can enter the n/w during time t is
max # of packets = c + t*r packets

So max rate = (c+tr)/t

$$\text{max \# of packets} = c + tr \qquad \text{max rate} = \frac{c+tr}{t}$$

Q. Consider a token bucket of capacity 250 KB and tokens arrive at a rate of 2 MBps.
If the maximum output rate is 25 MBps, what is the burst time?

→ $C = 250$ KB $r = 2$ MBps outflow = 25 MBps

$$\text{max rate} = \frac{C + Tr}{T} = 25$$

$T = 10.86$ ms.

The token bucket algorithm can be conceptually understood as follows:

- A token is added to the bucket every $1/r$ seconds.
- The bucket can hold at the most b tokens. If a token arrives when the bucket is full, it is discarded.
- When a packet (network layer PDU) of n bytes arrives,
  - if at least n tokens are in the bucket, n tokens are removed from the bucket, and the packet is sent to the network.
  - if fewer than n tokens are available, no tokens are removed from the bucket, and the packet is considered to be *non-conformant*.

UDP: User Datagram Protocol

TCP is heavy. It has a lot of functionalities. Many applications need a simple TL.

i. Say we're walking on the street and we do not know what the time is.
We ask someone, he replies 10:00 am.
1 request, 1 reply

This is the NTP: Network Time Protocol.

For this, TCP would establish a connection (3-way handshake), transfer data, close the connection (too much overhead).

Similarly there are similar applications:
- NTP
- BOOTP
- NNP: Network News Protocol
- DNS: Domain Name Service
- Quote of the day
...

All of these services need 1 req, 1 reply.

ii. Broadcasting and Multicasting:



say a sender has a lot of connections (2^34)
2^34 * 1MB = 16 GB
Too much resources are required.
TCP isn't suitable.

iii. Some applications require speed more than reliability. E.g. gaming or multimedia like YouTube.
In TCP, we have the AIMD algo. (slow start, congestion avoidance, drop,...
(AIMD is additive increase multiplicative decrease)
TCP doesn't provide speed; The data rate fluctuates. So it isn't needed.



UDP takes IP datagram from NL and gives it to the user at AL directly.
Essentially UDP means doing nothing at the TL.

There's no connection, so there's no SYN, no seq #s;
no acknowledgement, so no ACK flags, no ack #s;
no flow control, so no adv window;
no extra features like URG, RST, PSH, etc.
So the UDP header is quite simple.

UDP header:

source port (16)	destination port (16)
length (16)	checksum (16)

checksum is calculated on
UDP header + UDP data + pseudo IP header

Hardware Devices used in Networking:

Cables: (specifically Ethernet)

Ethernet cables are 10 base T, 10 base 2, 10 base 5, 100 base T

10 Base T

- 10 → Bandwidth in Mbps.
- Base → Baseband → Only 1 station can transmit (Broadband: many stations can transmit at the same time)
- T → 100 m (length of cable)
  - 2 : 200 m
  - 5 : 500 m
  - T : 100 m

Obv cables don't have TCP/IP layers.
They work at the physical layer.

Problem: Attenuation

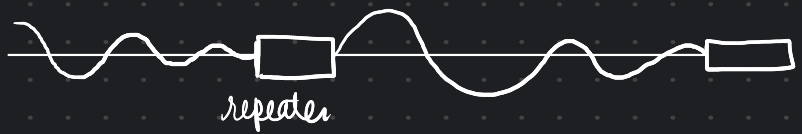
The stength of a signal decreases gradually.
So there's a limit on the length of the cable.

Problem: Collisions

Say n stations are connected. All n of them can collide with one another.
Collision domain = n

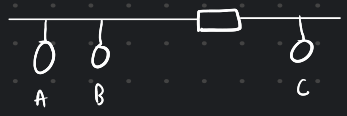
Repeater:

For a 10 Base T cable, the range is merely 100 metres. And there'll be attenuation.
So we need repeaters.



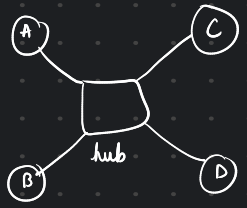
a repeater works in both directions

- used to connect 2 LAN segments; increases range of LAN
- works at the PL
- collisions are possible inside repeaters:
  connection domain = n
- unnecessary increase in traffic:



say A transmits to B
But transmission will be sent to C as well

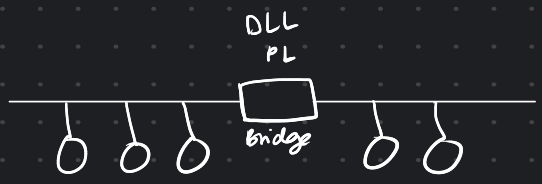
Hub: Multiport repeater



- only PL
- traffic is very high:
  a packet sent to A will be sent to B, C, D as well
  packets are flooded
- high collisions:
  collision domain = n (all can collide with each other)

Adv:
- it is cheap

Bridge:



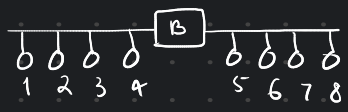
- connects different LAN segments
- has DLL and PL
- since it has DLL, it can see the MAC address:
  say A wants to send a message to B



the message will be filtered and forwarded

Bridges

static

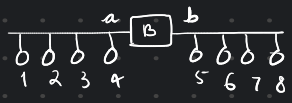


mapping table:

MAC	Port

It is static -
manually configured

dynamic/learning/transparent



MAC	interface

when 1 sends a packet, B will know
that 1 is connected to a.
So it'll make an entry (1,a)

say now 1 sends a packet to 3.
But B doesn't know 3.
In that case, it'll flood the packet.

bridge can filter, forward, or flood

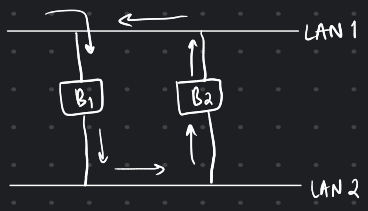
there aren't any collisions inside the bridge, so the collision domain is reduced:
there may be collisions between stations say between packets reaching the bridge,
but collisions won't happen via the bridge

bridges store and forward
so no collisions inside

filtering?
if 1 and 2 are on the same interface,
then the bridge won't forward packet
if 1 and 3 are on diff interfaces,
bridge will forward the packet
(if it knows; else it'll flood)

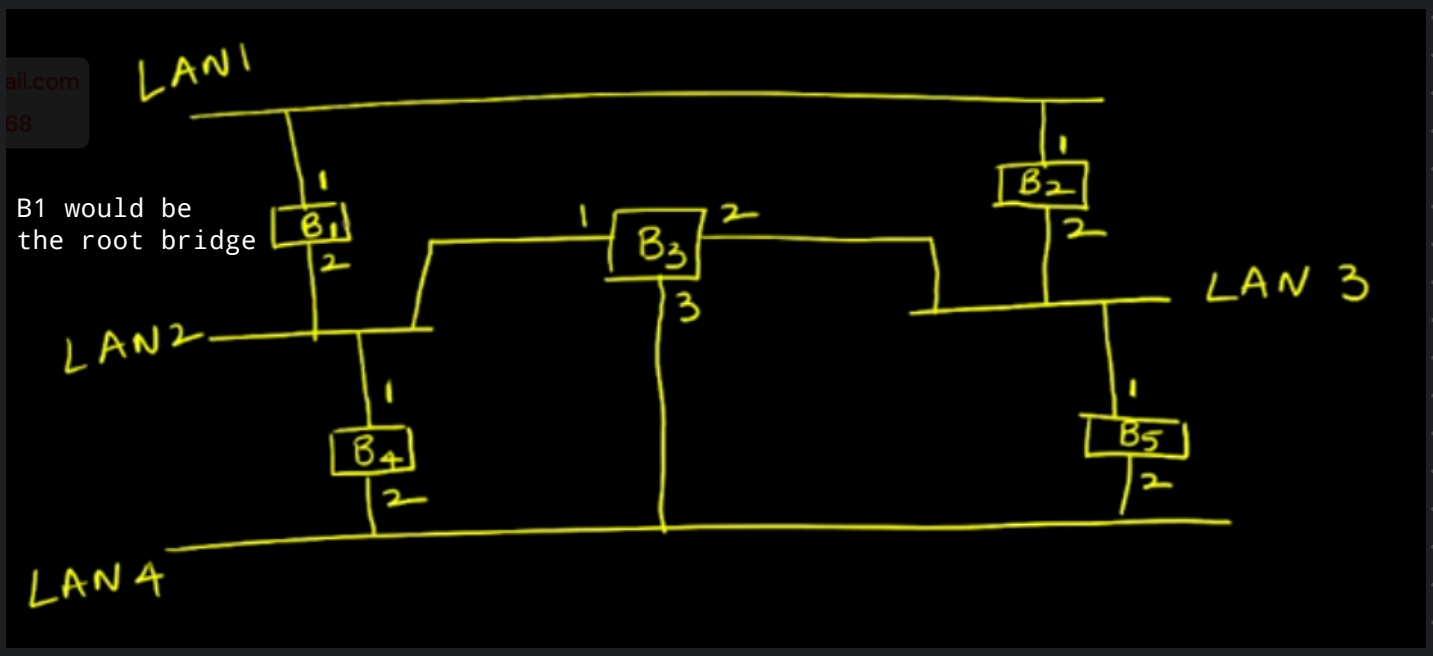
to increase reliability, more bridges will be added within a network.
if 1 bridge is down, some other bridge would be present.

disadv:
chance of loops
so we'll use the
spanning tree algo

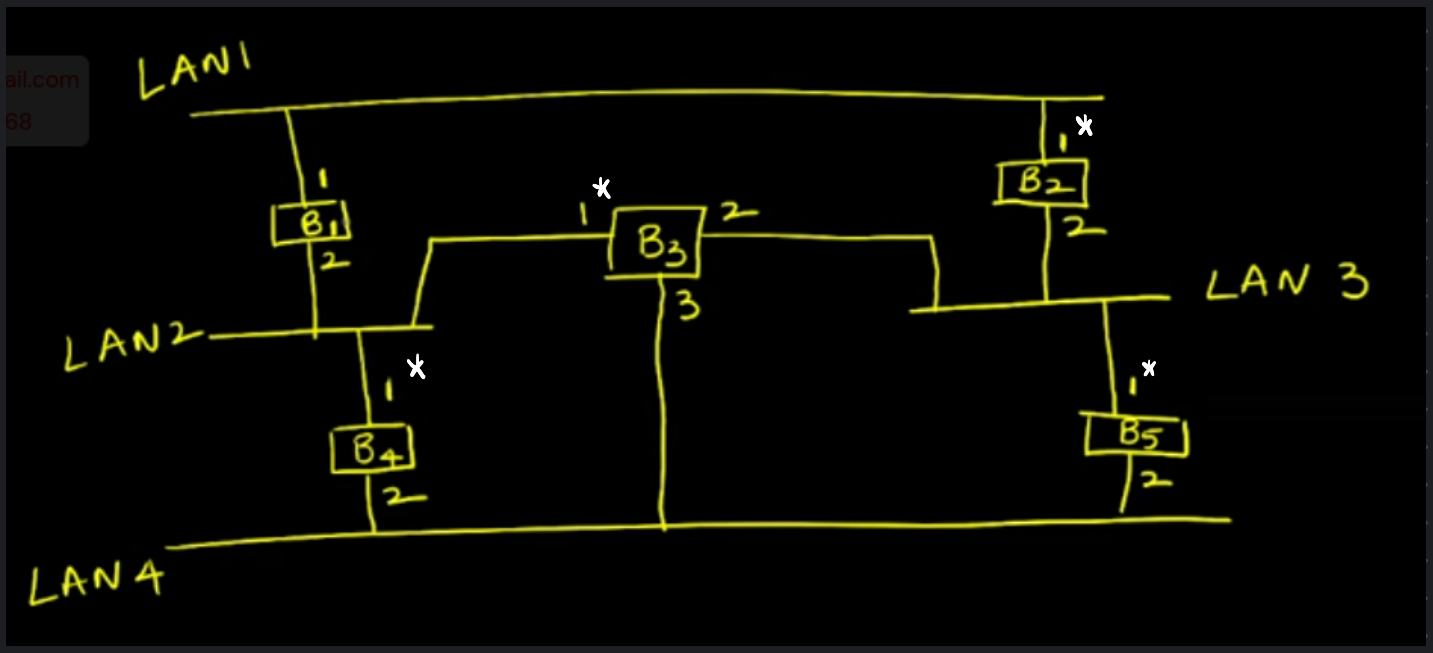


Spanning tree algo for bridges:

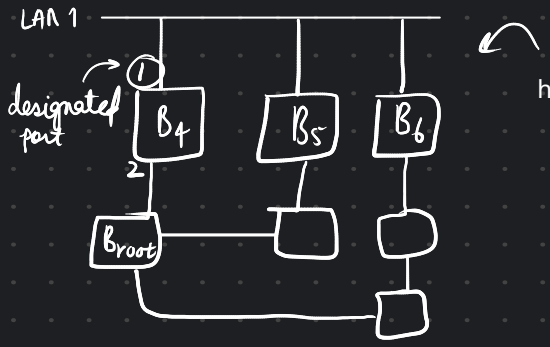
- Every bridge has a built-in ID. The one with the smallest ID is taken as the root bridge.
- mark 1 port of each bridge which is closest to the root bridge
- every LAN chooses the bridge close to root as the designated bridge for itself and assigns the corresponding port as the designated port
- mark the root port and the designated port as the forwarding ports and block the rest.



For B2, which port is closest to root? 1
For B3, 1. For B4, 1. For B5, either (let's go with the lowest #)

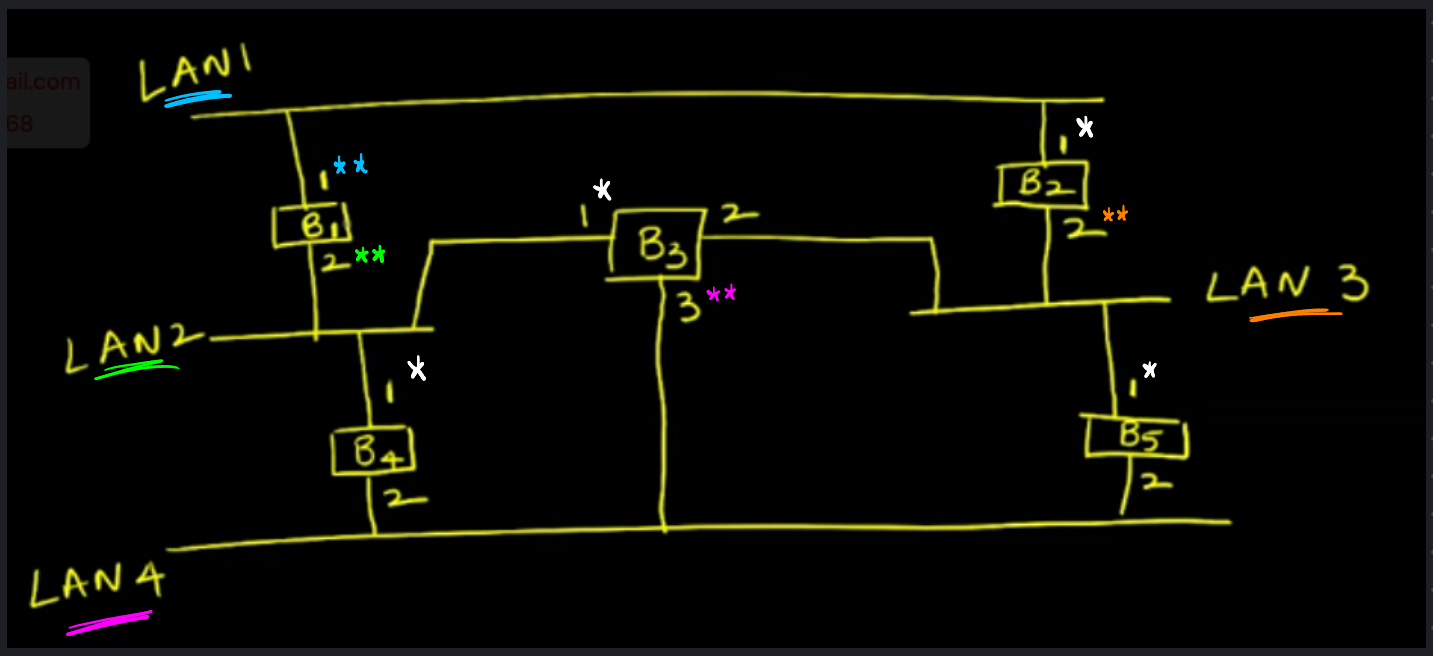


now choose bridge closest to root for each LAN, and the port connected to the LAN:

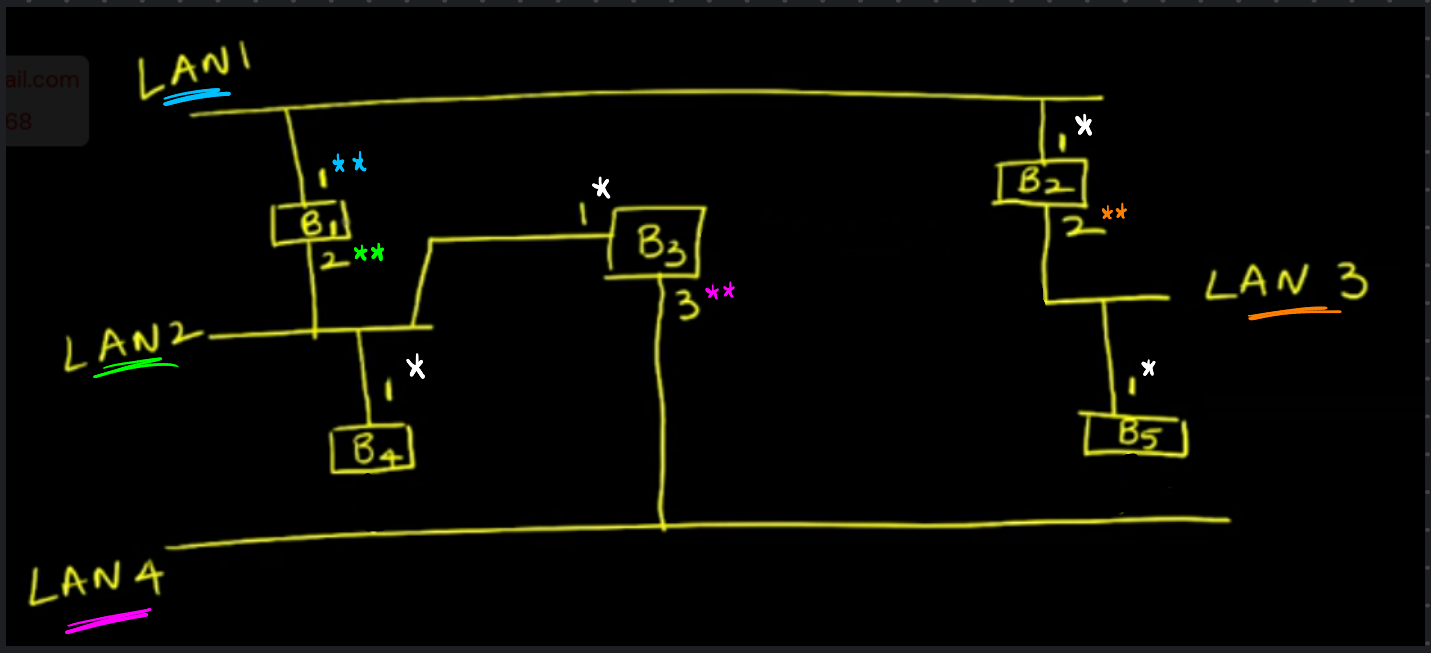


here, 1 will be the designated port for LAN 1

for LAN 1, B1 and B2 are connected. B1 is closer to the root (it is the root)
So port 1 of B1 will be the designated port. lly, the rest can be looked for.
For LAN 4, out of B3, B4, B5, we see that B3 and B4 are similarly close. We'll go with the smaller #: B3.

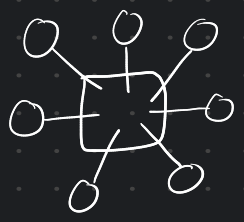


mark as forwarding and block the rest:



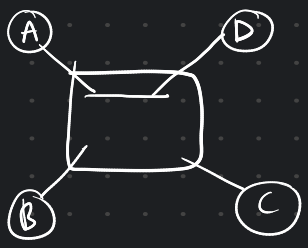
In this graph, there aren't any cycles.
This is the spanning tree.

Switches:



- PL and DLL
- can have 16, 32, 64, ports
- no collisions inside
  collision domain = 0
- like a bridge, but much more ports
- a switch can see MAC addresses,
  so traffic is reduced

Disadv:
- high cost



if A wants to send a message to D, a virtual connection betn them is established

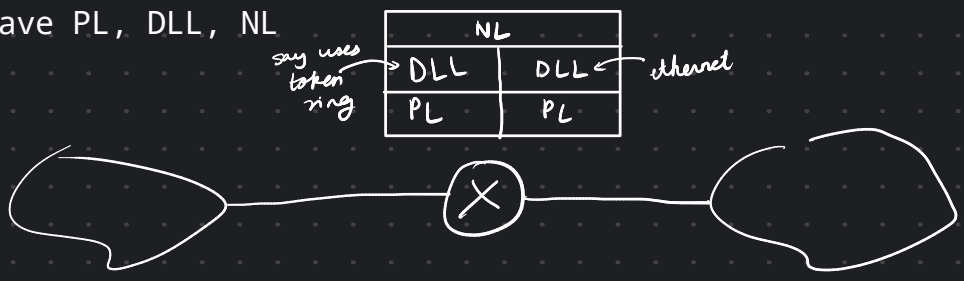
- Full duplex connections

Cables, hubs, repeaters, bridges, switches;
All of these are used within a network.
These cannot separate networks.
They will not stop broadcast packets.
So broadcast domain is not reduced.

Router:

- Routers can separate networks and create the internet.

- can have PL, DLL, NL



routers can connect networks using different protocols

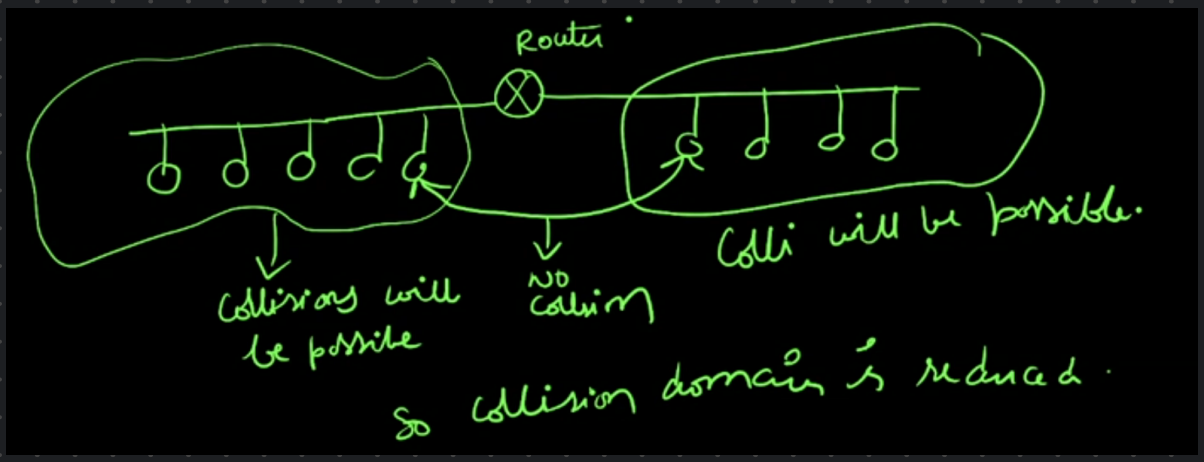
- routers can do forwarding, filtering, flooding, routing
- no collisions inside a router:
  collision domain is reduced

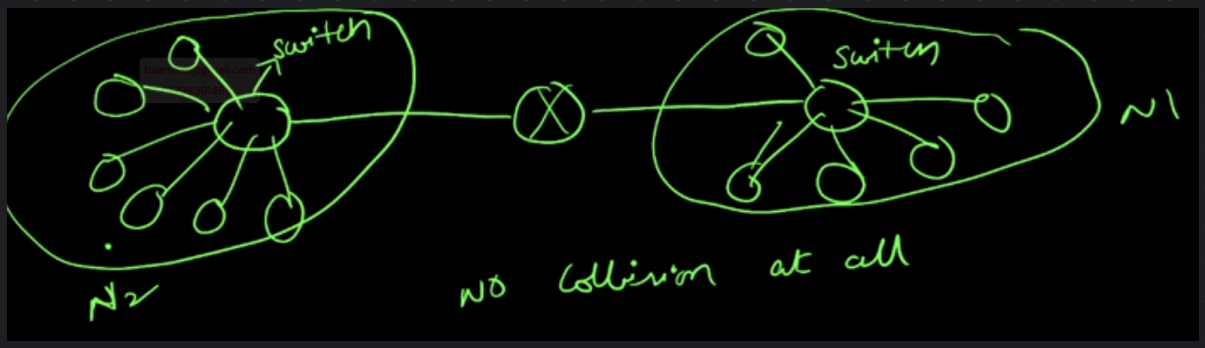
- routers stop broadcast packets (from going outside a network)
  so broadcast domain is reduced

A router can connect TCP/IP networks to TCP/IP networks;
ATM networks to ATM networks;

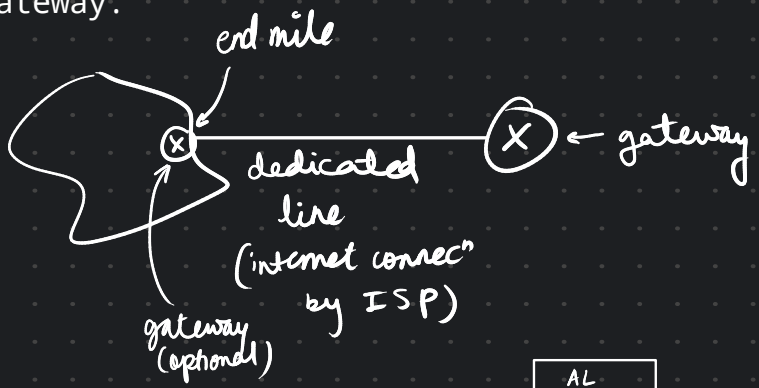
But cannot connect a TCP/IP network and an ATM network.
(as both have diff protocols at the NL,
and a router has only 1 NL)

ATM: Asynchronous Transfer Mode
is a protocol stack like TCP/IP





Gateway:



the ISP would provide a dedicated line and IP addresses



A gateway has all layers.
So it can connect n/ws with diff protocols at TL, NL, DLL, PL.
(so unlike a router, it'd be able to connect TCP/IP and ATM, or say ATM and X.25)

So gateways are also called protocol converters.

A gateway can act as proxy.
It can do caching and accounting.

say your phone is connected to YouTube

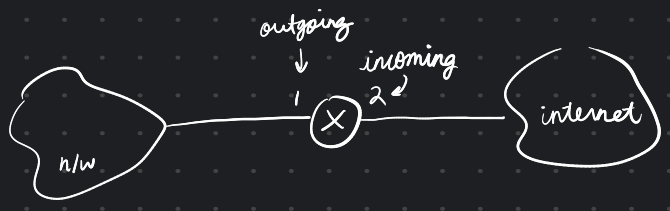
say YT sends data at 1Mbps,
but the phone can consume at 1Kbps.
The gateway will do buffering.

Say in your college, there's a rule that no student should use > 1 GB/week.
The student would be given credentials to login into the gateway (acting as proxy).
The proxy would measure the data and keep track of the data consumed.

A gateway can do NAT
A NAT gateway would substitute private IPs with a public IP while going out, and vice versa while coming in.

Firewall:

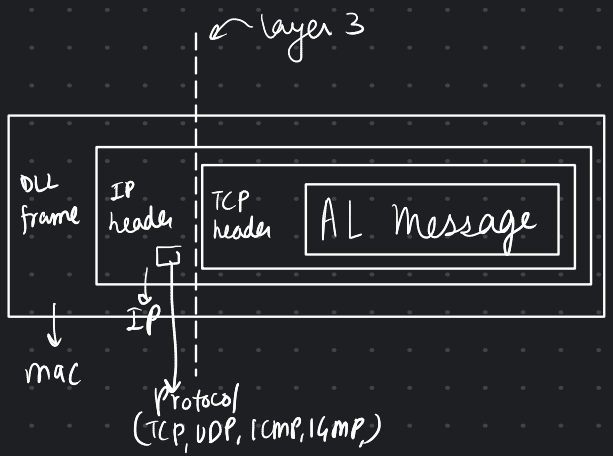
- protective stopping
- can block incoming or outgoing packets



Types of firewall:

- layer 3 firewall or packet filtering firewall (PL, DLL, NL)
- layer 4 firewall (PL, DLL, NL, TL)
- layer 5 firewall or proxy firewall (PL, DLL, NL, TL, AL)

Layer 3 firewall (PL, DLL, NL):

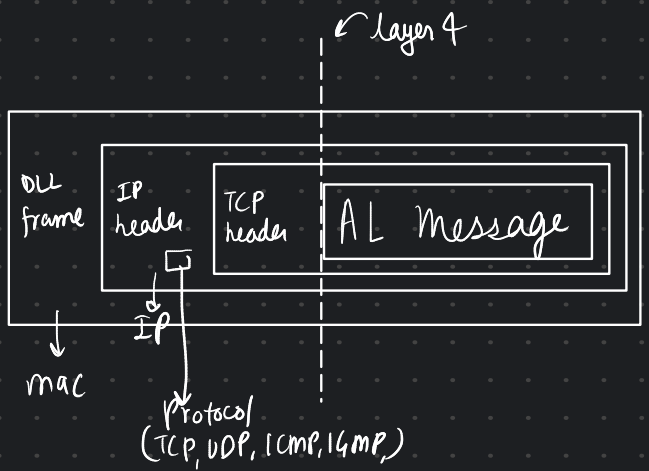


- it can block host (specific IP)
- it can block specific protocols (TCP, UDP,)
- it can block combinations (e.g. TCP from 100.1.2.3)

we don't block MAC addresses
(they are in the same network)

Layer 4 firewall (PL, DLL, NL, TL):

can do all the functionalities of layer 3 firewall and more



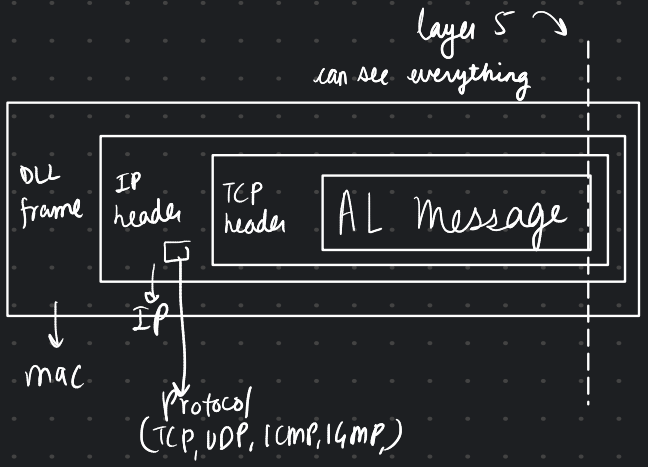
it can see port # as well

- so it can block services on specific ports
so it can block specific AL services

think of service as port and host as ip

Layer 5 firewall (proxy) (PL, DLL, NL, TL, AL):

can do all the functionalities of layer 4 firewall and more



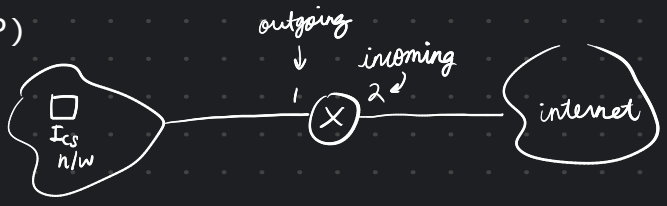
it can see everything including AL messages
(even usernames and passwords)

so it can block users, services, ports,
or any combination of these.

Generally a firewall will have a rule table

SIP	DIP	SP	DP	Interface	User
I_{CS}	—	21	—	1	—

such a rule would say that no packet
would go out from source port as 21 (FTP)
any such packet would be blocked



or say we don't want anyone to login into the code server I_{CS}
23 (Telnet) would be the dest port

SIP	DIP	SP	DP	Interface	User
—	I_{CS}	—	23	2	—

similarly we could block specific users, or other specific ports

---

Layer 5 or proxy firewalls are the most popular ones.
Layer 4 firewalls aren't used.
Layer 3 firewalls are used, though less commonly.

Application Layer Protocols: (DNS, HTTP, FTP, SMTP, POP)

DNS: Domain Name Service

- since IP addresses are not permanent or memorable, domain names are used
- so we need DNS to map domain names to IP addresses

Generic Domains:

.com, .edu, .mil, .org, .net, ...

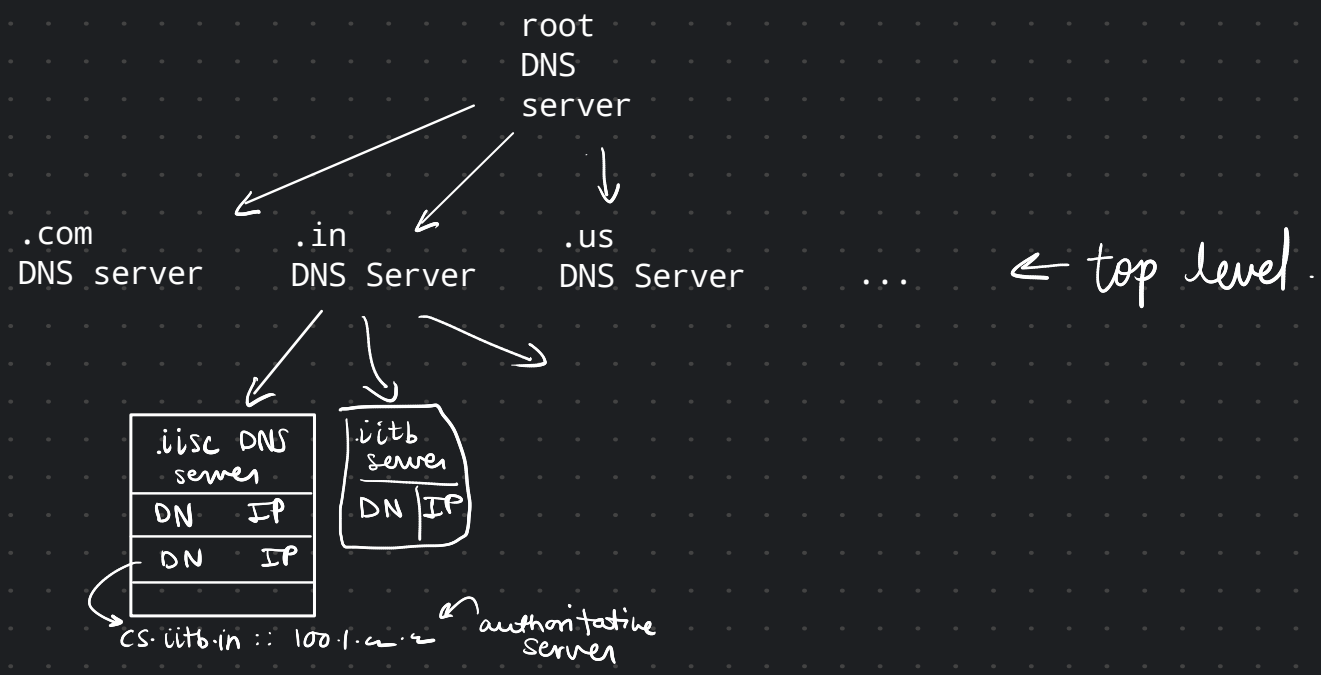
commercial education military non-profit network companies

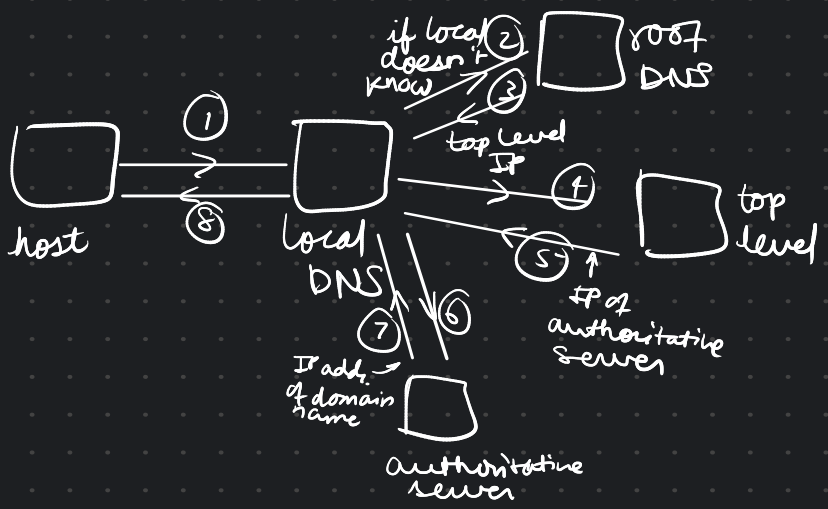
Country Domains:

.in, .us, .uk, .aus

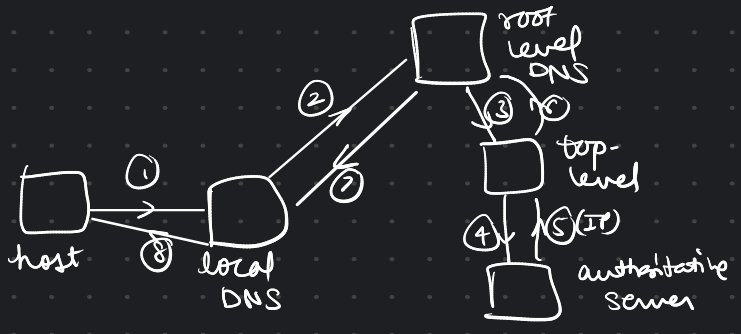
Inverse Domain: IP -> DN

mapping from IP to domain name instead of the other way





iterative approach



recursive approach

DNS uses UDP at TL. (single request, single reply)
DNS uses port # 53

HTTP: Hypertext Transfer Protocol

- used to get webpages
- needs reliability (so it uses TCP)
- port # = 80
- HTTP is inband protocol: both commands and data use the same connection
  (FTP has separate connections for commands and data: out-of-band)
- HTTP is stateless: doesn't remember anything about client

- HTTP 1.0 is non-persistent

Imagine there's 1 HTML file and 3 images: total 4 things to be downloaded

4 connections will be established
1 for each object

short connections

- HTTP 1.1 is persistent

1 connection will be established
for all of them

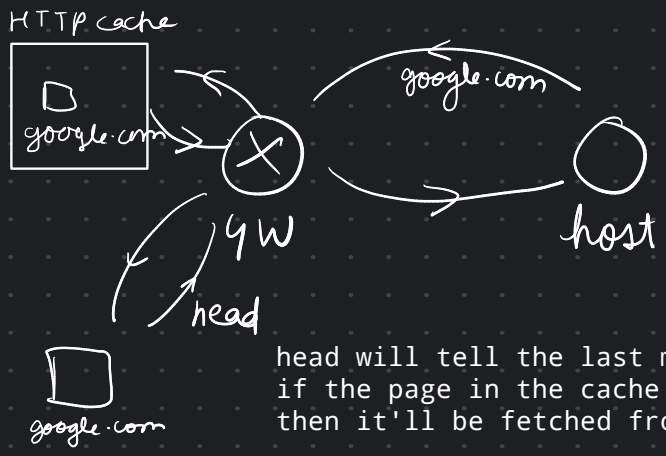
long connection

Methods supported by HTTP:

i. HEAD: gets metadata about the webpage
ii. GET: gets webpages
iii. POST: submits data (forms)
iv. PUT: uploads objects
v. DELETE: deletes objects
vi. TRACE: traces where the data is coming from
vii. OPTIONS: finds methods provided by the server
viii. CONNECT: not reqd

all of these may or may not be provided by a server

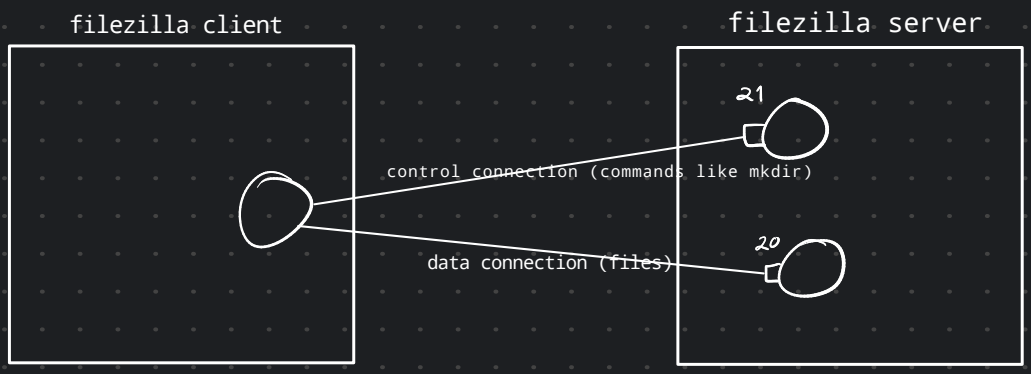
Generally, an ISP uses a web cache/ HTTP cache.



all ISPs do not use a web cache

head will tell the last modified time
if the page in the cache is the most recent one,
then it'll be fetched from the cache

FTP: File Transfer Protocol



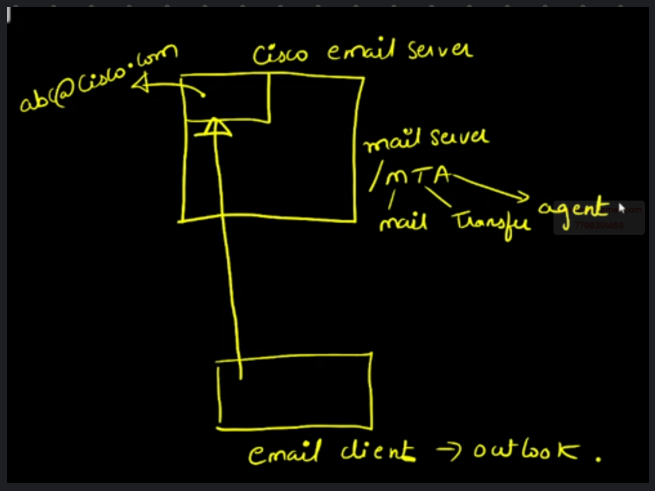
data connection is closed after every transfer (non-persistent)
control connection is always open (persistent)

FTP is out-of-band
FTP uses TCP
FTP is stateful: remembers clients

SMTP: Simple Mail Transfer Protocol
POP: Post Office Protocol

Initially, only few people had email ids.
Then hotmail gave email accounts to everyone.

An email account is essentially just a space in an email server.
If someone emails you, that email will be stored in your part of the email server.



say abc@cisco.com wants to send an email to xyz@ibm.com
It'll be sent to the cisco mail server, which will transfer it to the ibm mail server which will be read by an email client

sending to mail server: pushing
retrieval from server: pulling

pushing: SMTP
pulling: POP

- both SMTP and POP use TCP
- both are in-band
- port #s: STMP: 25 old, 587 new; POP: 110

Initially, emails were used only for text. Now non-text data is transferred as well.
To convert between text and non-text data, we use MIME.

MIME: Multipurpose Internet Mail Extension

Gmail is web-based.

